



Università degli Studi di Camerino

SCUOLA DI SCIENZE E TECNOLOGIE

Corso di Laurea in Informatica (Classe L-31)

**Progettazione e sviluppo di un framework
modulare per l'analisi ed elaborazione di flussi
video in Python**

Laureandi

Alejandro Innocenzi

Matteo Vittori

Matricole

130516

126646

Relatore

Michele Loreti

Correlatore

Vincenzo Fioriti

A.A. 2025/2026

Indice

1	Introduzione	11
1.1	Motivazione	12
1.2	Obiettivi	12
1.3	Approccio proposto	13
1.4	Contributi della tesi	14
1.5	Metodologia	14
2	Contesto, problema e requisiti	17
2.1	Motivazioni del progetto	17
2.2	Problema affrontato	19
2.3	Ambito e posizionamento del framework	20
2.4	Modello generale della pipeline	20
2.5	Requisiti funzionali	22
2.6	Requisiti non funzionali	23
2.7	Principali vincoli e scelte derivate dai requisiti	25
2.8	Casi d'uso di riferimento	26
2.9	Sintesi del capitolo	27
3	Architettura del framework	29
3.1	Architettura interna del framework	29
3.1.1	Flusso generale della pipeline	29
3.1.2	Separazione tra core e adattatori	31
3.1.3	Contratti e astrazioni principali	33
3.1.4	Plugin Registry e configurazione versionata	35
3.1.5	Pianificazione dell'esecuzione e runtime ibrido	36
3.1.6	Buffer, latenza e supporto al realtime	37
3.1.7	Eventi, branching e controllo real-time	38
3.2	Dettagli implementativi	40
3.2.1	Estrazione e processing dei frame	40

3.2.2	Estrazione, pulizia e analisi dei segnali	41
3.2.3	Visualizzazione, artifact e riproducibilit�	43
3.2.4	SEF Studio come strumento operativo	45
3.2.5	Modalit� d'uso	50
4	Tecnologie utilizzate	53
4.1	Gestione modulare delle dipendenze	53
4.2	Librerie per l'elaborazione video	54
4.2.1	OpenCV	54
4.2.2	NumPy	54
4.3	Modelli di computer vision	55
4.3.1	YOLO Pose	55
4.3.2	COCO Dataset	55
4.3.3	ArUco	56
4.4	Motion magnification	56
4.4.1	Eulerian	56
4.4.2	Phase-Based	57
4.5	Interfaccia grafica	57
4.5.1	Streamlit	57
4.5.2	Matplotlib	58
4.6	Strumenti di supporto	58
4.6.1	Ruff	58
4.6.2	PyTest	59
4.6.3	PyYAML	59
4.6.4	Git e GitHub	59
4.6.5	MkDocs	59
5	Casi d'uso, valutazione sperimentale	61
5.1	Video Tracking	61
5.1.1	Tracking a singolo oggetto	61
5.1.2	Tracking multi-oggetto	62
5.2	Optical flow	63
5.2.1	Sparse optical flow	64
5.2.2	Dense optical flow	64
5.3	Motion magnification	65
5.4	ArUco per displacement e moto relativo	65
5.5	COCO pose realtime e classificazione del gesto	66

5.6	Altri scenari applicativi	67
6	Benchmark e valutazione prestazionale	69
6.1	Streaming vs Batch	70
6.2	Valutazione delle execution policy	71
6.3	Overhead architetturale del framework	74
6.4	Scalabilità in scenari memory-bound	76
6.5	Discussione dei risultati	78
6.6	Conclusioni	78
7	Discussione finale	81
7.1	Validità del framework	81
7.2	Distribuzione e maturità del progetto	82
7.3	Limiti attuali	83
7.4	Sviluppi futuri e conclusioni	85

Elenco delle figure

2.1	Schema generale della pipeline SEF.	21
3.1	Regola delle dipendenze nella Clean Architecture.	31
3.2	Flusso concettuale di SEF Studio.	46
6.1	Tempo totale di esecuzione nelle diverse configurazioni batch e streaming.	70
6.2	Throughput medio espresso in frame al secondo (FPS).	71
6.3	Memoria massima utilizzata durante l'esecuzione della pipeline.	71
6.4	Tempo totale di esecuzione delle diverse execution policy.	72
6.5	Staleness dell'ultimo frame elaborato.	73
6.6	Latenza media di elaborazione.	73
6.7	Percentuale di frame scartati dalle execution policy.	74
6.8	Percentuale di frame effettivamente elaborati.	74
6.9	Rapporto di overhead rispetto a un'implementazione diretta.	75
6.10	Memoria massima utilizzata durante l'esecuzione della pipeline	76
6.11	Confronto del picco di memoria tra implementazione diretta, SEF batch e SEF streaming.	77
6.12	Rapporto di overhead rispetto a un'implementazione diretta al crescere della durata del video.	78

Elenco delle tabelle

2.1	Stage principali della pipeline SEF	21
2.2	Requisiti funzionali di SEF	22
2.3	Requisiti non funzionali di SEF	24
2.4	Sintesi delle principali scelte progettuali derivate dai requisiti	26
2.5	Casi d'uso di riferimento per la definizione dei requisiti	27

1. Introduzione

Negli ultimi anni l'analisi automatica di immagini e video ha assunto un ruolo sempre più rilevante in numerosi ambiti applicativi, tra cui monitoraggio, biomeccanica, controllo industriale, videosorveglianza, interazione uomo-macchina e analisi di fenomeni fisici. La crescente diffusione di telecamere, dispositivi mobili e sensori a basso costo ha reso possibile acquisire grandi quantità di dati visuali, aumentando al tempo stesso la complessità dei sistemi necessari per elaborarli in modo affidabile, riproducibile e adattabile a contesti differenti.

Un flusso video non rappresenta soltanto una sequenza di immagini, ma una sorgente da cui estrarre segnali, misure e informazioni derivate, come traiettorie, velocità, posture, relazioni tra oggetti o variazioni temporali di specifiche regioni di interesse. La trasformazione del dato visuale grezzo in informazioni utilizzabili richiede quindi una pipeline composta da più fasi di elaborazione, ciascuna responsabile della progressiva costruzione del risultato analitico.

Nella pratica, tali pipeline vengono spesso implementate tramite script monolitici basati su librerie come OpenCV, Matplotlib o framework dedicati all'inferenza di modelli di computer vision. Sebbene questo approccio risulti efficace nella realizzazione di prototipi rapidi o di singoli esperimenti, mostra diversi limiti quando il progetto cresce: duplicazione della logica, forte accoppiamento tra le diverse fasi dell'elaborazione, difficoltà di testing, scarsa riusabilità dei componenti e limitata possibilità di sostituire o estendere parti della pipeline senza intervenire sul resto del sistema. In contesti di ricerca, sperimentazione o didattica, tali criticità rendono più complesso confrontare approcci differenti e riprodurre gli esperimenti svolti.

Questa tesi presenta la progettazione e lo sviluppo di SEF (*Signal Extraction Framework*), un framework modulare in Python per l'analisi e l'elaborazione di flussi video. SEF nasce con l'obiettivo di fornire una struttura architetturale nella quale pipeline di computer vision possano essere costruite come composizione di componenti espliciti, indipendenti e sostituibili, mantenendo separati il nucleo del framework, le implementazioni concrete e le modalità di interazione con l'utente. Il framework non si propone di sostituire le librerie specializzate per l'elaborazione delle immagini o per l'inferenza di modelli, ma di coordinarne l'utilizzo all'interno di un modello architetturale coerente.

1.1 Motivazione

Lo sviluppo di applicazioni per l'analisi video richiede spesso l'integrazione di strumenti e tecnologie differenti. Una stessa pipeline può comprendere l'acquisizione dei frame, trasformazioni geometriche, stabilizzazione, rimozione del rumore, tracking di oggetti, rilevamento di marker, stima della posa, estrazione di segnali temporali e produzione di rappresentazioni grafiche o video annotati. Quando tali responsabilità vengono concentrate in un unico script, anche piccole modifiche tendono a propagarsi all'intero sistema.

Un esempio ricorrente è la sostituzione di un algoritmo di tracking con un'implementazione alternativa. In una pipeline non modulare, il tracking può essere strettamente intrecciato con il caricamento del video, la gestione dei parametri, il calcolo delle metriche e la visualizzazione dei risultati. Sostituire un algoritmo richiede quindi modifiche distribuite in più punti del codice, aumentando il rischio di regressioni. Analogamente, introdurre nuove modalità di visualizzazione o produrre artefatti intermedi può richiedere interventi invasivi quando i confini tra le responsabilità non sono chiaramente definiti.

La motivazione principale di SEF è affrontare questo problema architetturale. Il framework introduce contratti espliciti per le diverse fasi della pipeline e promuove la costruzione di componenti indipendenti, testabili e riutilizzabili. In questo modo diventa possibile separare il problema applicativo, ovvero ciò che si desidera analizzare o misurare, dagli aspetti infrastrutturali necessari per acquisire, orchestrare, monitorare e visualizzare il flusso di elaborazione.

1.2 Obiettivi

Il lavoro svolto per questa tesi, svolto nell'ambito del progetto SECURE, si concentra su obiettivi sia architetturali sia implementativi. Il primo obiettivo è definire un modello di pipeline chiaro, in cui ogni fase abbia una responsabilità precisa e comunichi con le altre attraverso tipi e interfacce esplicite. In questo modello, l'acquisizione dei frame, la trasformazione delle immagini, l'estrazione dei segnali, la pulizia dei dati, l'analisi e la visualizzazione sono trattate come componenti distinti.

Il secondo obiettivo è mantenere il nucleo del framework indipendente dalle librerie concrete utilizzate per elaborare i dati. Il core di SEF definisce contratti, artefatti, gestione degli errori, orchestrazione, registro dei componenti e politiche di esecuzione; le implementazioni basate su OpenCV, Matplotlib, modelli di posa o interfacce grafiche sono invece collocate in livelli esterni. Questa separazione permette di introdurre nuovi adattatori senza modificare le regole centrali del sistema.

Il terzo obiettivo è supportare diverse modalità di utilizzo. SEF deve poter essere usato come libreria Python, tramite una API fluente, ma anche attraverso configura-

zioni dichiarative e strumenti da linea di comando. Inoltre, il framework deve poter sostenere sia elaborazioni offline su video registrati, sia scenari più vicini al tempo reale, nei quali i risultati vengono prodotti in modo progressivo e possono essere visualizzati durante l'esecuzione.

Infine, il progetto mira a offrire un insieme iniziale di componenti pronti all'uso, utile per validare l'architettura e per costruire casi d'uso concreti. Tra questi rientrano estrattori di frame, processori basati su OpenCV, estrattori di segnali, componenti di pulizia, analizzatori, visualizzatori ed esportatori di artefatti intermedi.

1.3 Approccio proposto

L'approccio adottato segue i principi della modularità e della separazione delle responsabilità. SEF organizza il dominio applicativo attorno al concetto di pipeline, intesa come composizione ordinata di componenti. Ogni componente espone un contratto: riceve un tipo di dato previsto, produce un risultato esplicito e può dichiarare capacità aggiuntive, come il supporto all'esecuzione streaming o la compatibilità con scenari in tempo reale.

Il core del framework contiene le astrazioni principali: interfacce dei componenti, artefatti di dominio, runtime di esecuzione, gestione degli eventi, registro dei plugin, errori tipizzati e oggetti di output. Le dipendenze verso librerie esterne sono mantenute fuori dal core, all'interno di moduli built-in o di eventuali plugin utente. In questo modo le regole centrali del framework non dipendono da dettagli infrastrutturali, come il backend di visualizzazione o la libreria scelta per leggere i video.

La pipeline può essere costruita in modo programmatico, usando una interfaccia Python orientata alla composizione, oppure descritta tramite file di configurazione. Nel primo caso, l'utente può sfruttare direttamente classi, funzioni e componenti registrati; nel secondo, può definire un flusso riproducibile e validabile tramite strumenti automatici. Il registro dei plugin ha il compito di associare nomi, categorie e metadati alle implementazioni disponibili, rendendo possibile l'ispezione dei componenti e la costruzione dinamica delle pipeline.

Un aspetto centrale del progetto è la distinzione tra elaborazione batch ed elaborazione streaming. Nel caso batch, l'intera sequenza di dati può essere processata prima di passare alla fase successiva. Nel caso streaming, invece, alcuni componenti possono produrre risultati progressivi man mano che i frame vengono acquisiti. SEF introduce contratti e politiche di esecuzione per gestire entrambe le modalità, mantenendo un modello concettuale uniforme.

1.4 Contributi della tesi

Il contributo principale di questa tesi è la realizzazione di un framework modulare per la costruzione di pipeline video-to-signal. Tale contributo si articola in più risultati concreti:

- definizione di un'architettura a livelli, con un core indipendente da librerie esterne e adattatori concreti collocati nei moduli periferici;
- progettazione di contratti espliciti per frame extractor, frame processor, signal extractor, signal cleaner, analyzer, visualizer ed exporter;
- implementazione di un registro dei componenti e di un meccanismo di plugin per estendere il framework senza modificare il core;
- supporto a pipeline configurabili tramite API Python e tramite file di configurazione validabili;
- introduzione di un runtime capace di gestire esecuzioni batch, streaming, artefatti intermedi, eventi di ciclo di vita e output UI-agnostici;
- realizzazione di componenti built-in basati su tecnologie diffuse, come OpenCV e Matplotlib, per validare l'usabilità del modello proposto;
- sviluppo di strumenti di supporto, tra cui comandi CLI e una interfaccia sperimentale per comporre, eseguire e ispezionare pipeline.

Questi elementi permettono di utilizzare SEF sia come strumento didattico per studiare architetture modulari di computer vision, sia come base sperimentale per costruire analisi video riproducibili e facilmente estendibili.

1.5 Metodologia

La metodologia seguita combina analisi architetturale, sviluppo incrementale e validazione tramite casi d'uso. In una prima fase sono state individuate le responsabilità ricorrenti nelle pipeline di elaborazione video e sono stati definiti i confini tra core, componenti applicativi e adattatori infrastrutturali. Successivamente sono stati progettati i contratti pubblici e i tipi di dato necessari a rappresentare frame, segnali, risultati analitici e artefatti di visualizzazione.

L'implementazione è stata guidata dalla necessità di mantenere il framework estensibile. Per questo motivo, le funzionalità concrete sono state introdotte come componenti registrabili, evitando di incorporare nel runtime dipendenze specifiche da una singola libreria o da un singolo caso d'uso. Ogni nuova fase della pipeline è stata progettata in modo da poter essere testata isolatamente e sostituita con implementazioni alternative.

La validazione del lavoro è avvenuta attraverso pipeline dimostrative e casi d'uso orientati all'analisi del movimento, al tracking di oggetti, al rilevamento di marker ArUco, motion magnification e alla produzione di risultati visuali [Wu+12]. Questi scenari hanno permesso di verificare che il modello proposto fosse sufficientemente generale da coprire elaborazioni diverse, ma anche abbastanza strutturato da imporre confini chiari tra le responsabilità.

2. Contesto, problema e requisiti

Questo capitolo introduce il contesto applicativo e progettuale nel quale è stato sviluppato SEF, chiarendo il problema affrontato e i requisiti che hanno guidato le successive scelte architetturali. L'obiettivo non è descrivere nel dettaglio l'implementazione del framework, ma motivare le esigenze da cui essa deriva: modularità, estendibilità, riproducibilità, osservabilità e separazione tra logica applicativa e tecnologie concrete.

L'elaborazione di flussi video rappresenta un ambito rilevante sia in contesti scientifici sia in applicazioni industriali. Tecniche di *computer vision*, elaborazione dei segnali e *machine learning* permettono oggi di estrarre informazioni significative da video grezzi, rendendo possibile affrontare problemi come il tracciamento di oggetti, l'analisi del movimento, la stima della posa umana, il monitoraggio strutturale, il conteggio di attraversamenti e l'individuazione di fenomeni difficilmente percepibili a occhio nudo.

Tuttavia, la costruzione di sistemi completi per l'analisi video richiede spesso l'integrazione manuale di strumenti eterogenei. Librerie come OpenCV, modelli di *pose estimation*, strumenti di visualizzazione, moduli per la gestione di file video e componenti per l'elaborazione numerica risolvono problemi specifici, ma non forniscono necessariamente una struttura comune per coordinare l'intero processo. Di conseguenza, molte soluzioni vengono realizzate come script monolitici o prototipi fortemente dipendenti dal singolo caso d'uso.

Questa impostazione può essere efficace nelle prime fasi sperimentali, ma diventa problematica quando il sistema deve essere riutilizzato, esteso, testato o adattato a scenari differenti. SEF nasce per rispondere a questa esigenza, proponendo un modello comune nel quale le diverse fasi dell'elaborazione video possano essere organizzate, sostituite e combinate in modo modulare.

2.1 Motivazioni del progetto

Il progetto SEF è nato nell'ambito di un'attività di stage svolta sotto la supervisione del prof. Michele Loreti. Nelle prime fasi sono state valutate diverse direzioni progettuali, con l'obiettivo di individuare un tema tecnicamente significativo, ad esempio, legato ad ambiti attuali come l'intelligenza artificiale e il *machine learning*, ma anche sufficientemente ampio da consentire lo sviluppo di un sistema software non limitato a

un singolo esperimento.

Tra le alternative considerate, la progettazione di un framework modulare per l'analisi e l'elaborazione di flussi video è risultata la scelta più adatta. Questa direzione permetteva infatti di combinare un'esigenza applicativa concreta con un obiettivo più generale di progettazione software: costruire un'infrastruttura riutilizzabile, capace di supportare più casi d'uso senza vincolare l'intero sistema a una specifica libreria o a uno specifico algoritmo.

L'idea iniziale era collegata alla *motion magnification*, tecnica che permette di evidenziare micro-movimenti difficilmente percepibili a occhio nudo [Wu+12]. Uno scenario di questo tipo può risultare utile, ad esempio, nel monitoraggio strutturale, dove piccole oscillazioni o deformazioni possono fornire informazioni rilevanti sul comportamento di una struttura. Questo caso d'uso ha evidenziato fin da subito la necessità di distinguere il video grezzo dai segnali temporali estraibili da esso.

Il primo caso sperimentale affrontato durante lo sviluppo è stato però più semplice: il tracciamento di oggetti in movimento tramite OpenCV. Questa scelta ha permesso di validare le prime ipotesi implementative e costruire una bozza iniziale dell'architettura. Anche un caso relativamente semplice come il tracking ha mostrato rapidamente un problema fondamentale: per uno stesso obiettivo possono esistere più librerie, più algoritmi e più strategie implementative.

Una soluzione costruita direttamente attorno a una singola libreria avrebbe reso il sistema rigido e difficilmente riutilizzabile. Cambiare algoritmo di tracking, sostituire un modulo di analisi o introdurre una diversa tecnica di visualizzazione avrebbe richiesto modifiche distribuite in più parti del codice. Questa criticità ha motivato la scelta di progettare SEF come un framework modulare, basato su componenti indipendenti e contratti espliciti.

La decisione architetturale principale è stata quindi la definizione degli stage della pipeline, dei dati prodotti e consumati da ciascuno stage e dei contratti che ogni componente deve rispettare. Invece di considerare l'elaborazione video come un unico blocco indivisibile, SEF la scompone in fasi distinte, ciascuna associata a una responsabilità specifica.

Questa separazione consente di distinguere il dato visuale di partenza dall'informazione ricavata. Un video contiene inizialmente frame, pixel e sequenze temporali; il framework deve permettere di trasformare questi elementi in segnali, misure, risultati analitici e artefatti visuali. Tale distinzione è importante perché la stessa sorgente video può essere analizzata con obiettivi differenti: ad esempio la traiettoria di un oggetto, la variazione di un segnale nel tempo, la produzione di un video annotato o la generazione di un grafico riassuntivo.

Lo sviluppo del progetto è avvenuto seguendo un approccio iterativo e incrementale. Le funzionalità sono state introdotte progressivamente, a partire da prototipi semplici fino ad arrivare a una struttura più generale e riutilizzabile. Questa modalità di lavoro

ha permesso di correggere le scelte progettuali sulla base dei problemi emersi durante le sperimentazioni e di far evolvere il progetto da una soluzione orientata a un singolo caso d'uso a un framework più generale.

A posteriori, uno degli aspetti più delicati del progetto riguarda il bilanciamento tra due esigenze: da un lato la volontà di supportare casi d'uso concreti, dall'altro la necessità di costruire un'infrastruttura modulare, estendibile e indipendente dal singolo scenario applicativo. Questa tensione progettuale ha avuto un ruolo positivo, poiché ha spinto il framework verso una struttura sufficientemente generale da poter supportare elaborazioni diverse.

2.2 Problema affrontato

Il problema affrontato da SEF non consiste semplicemente nell'elaborare un video, ma nel fornire una struttura comune per organizzare l'intero ciclo di elaborazione. In molti progetti di analisi video, il flusso di lavoro segue una sequenza ricorrente: acquisizione dei frame, trasformazione del dato visuale, estrazione di segnali, eventuale pulizia dei dati, analisi, visualizzazione ed esportazione dei risultati.

Senza un'infrastruttura comune, queste fasi tendono a essere implementate direttamente all'interno dello stesso script. Questa soluzione può essere rapida durante la prototipazione, ma porta a diversi problemi:

- forte accoppiamento tra logica applicativa, librerie esterne e formato degli output;
- difficoltà nel sostituire una libreria o un algoritmo senza modificare il resto del codice;
- assenza di una rappresentazione esplicita della pipeline;
- scarsa riusabilità dei componenti sviluppati;
- difficoltà nel riprodurre esperimenti o confrontare configurazioni differenti;
- gestione poco strutturata degli errori e dei risultati prodotti;
- limitata osservabilità durante l'esecuzione, soprattutto in pipeline lunghe o progressive.

SEF affronta queste criticità proponendo un modello basato su pipeline modulari. Ogni pipeline è composta da stage con responsabilità specifiche e interfacce definite. In questo modo, il framework non impone un singolo algoritmo o una singola modalità di analisi, ma fornisce una struttura nella quale componenti diversi possono essere collegati tra loro in modo coerente.

Il concetto centrale è quello di *signal extraction*: a partire da un flusso video grezzo, SEF mira a estrarre segnali, dati e informazioni analitiche riutilizzabili. Il video non

viene quindi trattato soltanto come contenuto visuale da mostrare, ma come sorgente di dati temporali da elaborare. Questa impostazione consente di applicare lo stesso schema concettuale a scenari differenti, come object tracking, optical flow, rilevamento di marker ArUco, analisi della posa, *motion magnification* e monitoraggio di eventi.

2.3 Ambito e posizionamento del framework

SEF non nasce come alternativa diretta alle librerie di *computer vision* esistenti. Al contrario, il framework è progettato per coordinarle e renderle più facilmente integrabili all'interno di pipeline complesse. Librerie come OpenCV, strumenti di visualizzazione, modelli di *machine learning* o implementazioni basate su Ultralytics YOLO rimangono componenti fondamentali, ma vengono collocati in un'architettura più ampia.

In questa prospettiva, SEF può essere visto come un livello di orchestrazione e organizzazione. Le librerie esterne si occupano dell'esecuzione di algoritmi specifici, mentre il framework definisce il modo in cui tali algoritmi vengono composti, configurati, eseguiti e osservati. Questa separazione è essenziale per evitare che il nucleo del sistema dipenda direttamente da tecnologie specifiche come OpenCV, Matplotlib, Ultralytics o Streamlit.

Il core del framework contiene contratti, modelli dati, runtime, planner, registro dei plugin, gestione degli eventi, errori tipizzati e sistema di artefatti. Le implementazioni concrete, invece, vengono collocate in moduli built-in o in adapter esterni. Questa scelta riduce l'accoppiamento e consente di mantenere il nucleo del framework stabile anche quando cambiano le librerie utilizzate nei singoli casi d'uso. Va comunque precisato che il core non è completamente privo di dipendenze esterne, poiché utilizza NumPy per rappresentare alcuni dati fondamentali, come i frame video.

Dal punto di vista dell'utente, SEF può essere utilizzato in modi diversi. Un programmatore può costruire pipeline tramite API Python o plugin personalizzati; un utente più operativo può eseguire configurazioni dichiarative tramite file YAML o JSON; un utente meno esperto può interagire con SEF Studio, l'interfaccia grafica realizzata con Streamlit. Questa pluralità di modalità d'uso ha influenzato direttamente la definizione dei requisiti.

2.4 Modello generale della pipeline

Il modello di elaborazione adottato da SEF si basa sulla scomposizione del flusso video in una sequenza di stage. Ogni stage svolge una responsabilità specifica e comunica con gli altri attraverso dati strutturati. La pipeline generale può essere descritta dal seguente diagramma:

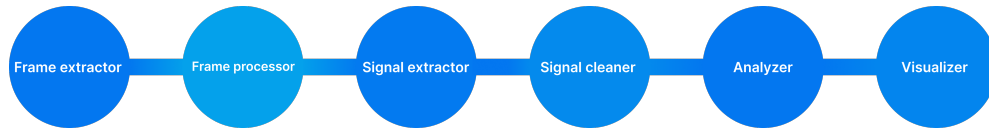


Figura 2.1: Schema generale della pipeline SEF.

Il *frame extractor* acquisisce frame da una sorgente video, come un file, una webcam o un flusso continuo. I *frame processor* applicano trasformazioni intermedie al dato visuale, ad esempio filtraggio, stabilizzazione, preprocessing o generazione di maschere. I *frame exporter* producono output collegati ai frame elaborati, come immagini, video o artefatti intermedi.

La fase di *signal extraction* trasforma il dato visuale in segnali o misure. Un componente di questo tipo può estrarre, ad esempio, la posizione di un oggetto, il movimento di un marker, una traiettoria, una distanza relativa o una serie temporale. I *signal cleaner* applicano operazioni di pulizia, normalizzazione o filtraggio ai dati estratti. Gli *analyzer* interpretano i segnali e producono risultati analitici, mentre i *visualizer* generano rappresentazioni visuali o testuali a partire dai risultati compatibili con la loro implementazione.

Questa struttura permette di separare il dato dalla sua interpretazione. Il frame rappresenta il dato visuale di partenza; il segnale rappresenta una misura estratta; il risultato analitico rappresenta un'informazione derivata; l'artefatto rappresenta infine una forma visualizzabile, esportabile o ispezionabile del risultato.

Tabella 2.1: Stage principali della pipeline SEF

Stage	Input/Output principali	Responsabilità
Frame extractor	Da video a frame	Acquisisce i frame da file, webcam o flussi video.
Frame processor	Da frame a frame elaborati	Applica trasformazioni intermedie al dato visuale.
Frame exporter	Da frame elaborati a output video/immagini	Produce output collegati ai frame o agli artefatti intermedi.
Signal extractor	Da frame o frame elaborati a segnali	Estrae misure, traiettorie o serie temporali dal video.
Signal cleaner	Da segnali a segnali elaborati	Filtra, normalizza o corregge i segnali estratti.
Analyzer	Da Segnali a dati	Interpreta i dati e produce informazioni analitiche.
Visualizer	Da dati ad artefatti	Genera grafici, immagini, video, tabelle o output testuali.

La scelta di suddividere l'elaborazione in stage indipendenti è direttamente collegata agli obiettivi di modularità ed estendibilità. Ogni componente può essere sostituito con

un altro compatibile con lo stesso contratto, senza richiedere la riscrittura dell'intera pipeline. Questo aspetto è particolarmente importante in un dominio, come quello dell'analisi video, nel quale gli algoritmi evolvono rapidamente e le librerie disponibili possono variare in base allo scenario applicativo.

2.5 Requisiti funzionali

Sulla base del problema affrontato e degli obiettivi progettuali, sono stati individuati i requisiti funzionali del framework. Essi descrivono le funzionalità che SEF deve fornire per supportare la costruzione, l'esecuzione e l'estensione di pipeline di elaborazione video.

Tabella 2.2: Requisiti funzionali di SEF

ID	Requisito	Descrizione	Evidenza progettuale
RF1	Costruzione di pipeline modulari	Il framework deve permettere di comporre pipeline costituite da stage indipendenti e sostituibili.	Contratti di stage, pipeline context.
RF2	Elaborazione video batch	Il sistema deve supportare l'elaborazione di video offline, utile per esperimenti riproducibili e analisi complete.	Runtime batch, output finali.
RF3	Supporto realtime e streaming	Il framework deve consentire l'elaborazione di flussi continui, includendo scenari di preview realtime.	Runtime streaming, capability degli stage.
RF4	Estrazione di segnali	SEF deve trasformare frame video grezzi in segnali, misure o dati strutturati.	Signal extractor.
RF5	Pulizia e trasformazione dei dati	Il sistema deve supportare operazioni di filtraggio, normalizzazione o correzione sui segnali estratti.	Signal cleaner.
RF6	Analisi dei risultati	Il framework deve permettere di interpretare i segnali e produrre risultati analitici.	Analyzer.
RF7	Visualizzazione ed esportazione	Il sistema deve produrre output visualizzabili o esportabili, come immagini, video, grafici, tabelle o JSON.	Artifact system, visualizer.

Continua nella pagina successiva

ID	Requisito	Descrizione	Evidenza progettuale
RF8	Configurazione dichiarativa	Le pipeline devono poter essere definite tramite file YAML o JSON, così da rendere le esecuzioni replicabili.	Config versionata.
RF9	Registro dei componenti	Il framework deve fornire un meccanismo per registrare, risolvere e istanziare componenti disponibili.	Plugin Registry.
RF10	Plugin personalizzati	Gli utenti devono poter aggiungere componenti custom tramite classi, funzioni o decorator.	Decorator e function adapters.
RF11	Interfaccia a riga di comando	Il framework deve poter essere usato da terminale per validare, eseguire e ispezionare pipeline.	CLI.
RF12	Interfaccia grafica	Il sistema deve offrire una modalità d'uso più accessibile tramite interfaccia visuale.	SEF Studio.
RF13	Osservazione dell'esecuzione	L'utente deve poter comprendere lo stato di una pipeline, gli eventi generati e gli output prodotti.	Eventi, monitor, execution plan.
RF14	Gestione strutturata degli errori	Il framework deve distinguere errori di configurazione, risoluzione plugin, costruzione ed esecuzione.	Error model tipizzato.

I requisiti funzionali evidenziano come SEF non si limiti a fornire singoli algoritmi di elaborazione video, ma punti a costruire un ambiente per definire, eseguire, osservare ed estendere pipeline. In particolare, l'estendibilità tramite plugin rappresenta un requisito centrale, poiché consente al framework di adattarsi a casi d'uso non previsti inizialmente.

2.6 Requisiti non funzionali

Accanto ai requisiti funzionali, sono stati definiti alcuni requisiti non funzionali. Essi descrivono le qualità architetturali che il framework deve possedere per risultare effettivamente utile, manutenibile ed evolvibile.

Tabella 2.3: Requisiti non funzionali di SEF

ID	Requisito	Descrizione	Soluzione adottata
RNF1	Modularità	Il sistema deve essere suddiviso in componenti con responsabilità distinte.	Pipeline a stage.
RNF2	Basso accoppiamento	Il core non deve dipendere direttamente da specifiche librerie di computer vision, visualizzazione o interfaccia utente.	Separazione tra core, built-in e adapter.
RNF3	Estendibilità	Deve essere possibile introdurre nuovi componenti senza modificare il nucleo del framework.	Plugin Registry e contratti.
RNF4	Manutenibilità	Il codice deve essere organizzato in modo da facilitare modifiche, test e correzioni.	Separazione delle responsabilità.
RNF5	Usabilità	Il framework deve poter essere utilizzato attraverso più modalità di accesso, in base al profilo dell'utente.	API, CLI e SEF Studio.
RNF6	Riproducibilità	Le esecuzioni devono poter essere replicate a partire da una configurazione esplicita.	YAML/JSON versionati.
RNF7	Osservabilità	Il comportamento della pipeline deve essere spiegabile e monitorabile.	Execution plan, eventi e monitor.
RNF8	Scalabilità architetturale	La struttura deve supportare l'aggiunta progressiva di nuovi casi d'uso e nuove librerie.	Adapter e plugin.
RNF9	Gestione controllata delle risorse	Il sistema deve considerare memoria, latenza e costo computazionale, soprattutto in pipeline lunghe o progressive.	Buffer bounded e latency policies.
RNF10	Portabilità degli output	Gli output non devono essere vincolati a una singola interfaccia grafica o a un singolo ambiente di esecuzione.	VisualArtifact e PipelineOutputs.
RNF11	Diagnostica robusta	Gli errori devono essere gestiti in modo strutturato e comprensibile da API, CLI e interfacce grafiche.	Errori tipizzati.
RNF12	Evolubilità della configurazione	Lo schema delle configurazioni deve poter evolvere senza rompere immediatamente le pipeline esistenti.	Schema versionato.

Tra questi requisiti, modularità, estendibilità e osservabilità assumono un ruolo particolarmente importante. Un framework di questo tipo non deve soltanto eseguire cor-

rettamente una pipeline, ma deve permettere agli utenti di comprenderne la struttura, sostituire componenti, diagnosticare errori e riprodurre i risultati ottenuti.

2.7 Principali vincoli e scelte derivate dai requisiti

I requisiti individuati hanno portato a una serie di scelte progettuali che verranno approfondite nel capitolo successivo. In questa sezione esse vengono introdotte a livello generale, con l'obiettivo di collegare il problema iniziale alle soluzioni architetturali adottate.

Un primo vincolo riguarda la necessità di evitare che il core del framework dipenda direttamente da librerie come OpenCV, Matplotlib, Ultralytics o Streamlit. Queste tecnologie sono utili e, in alcuni casi, fondamentali per implementare specifici componenti, ma non devono determinare la struttura interna del framework. Per questo motivo, SEF separa il nucleo logico dalle implementazioni concrete e dagli adapter.

Un secondo vincolo riguarda la necessità di supportare modalità d'uso differenti. La stessa pipeline deve poter essere costruita tramite codice Python, configurazione YAML/JSON, CLI o interfaccia grafica. Per evitare duplicazioni, queste modalità convergono verso un modello comune di pipeline, che rappresenta la struttura da eseguire indipendentemente dal punto di ingresso scelto dall'utente.

Un terzo vincolo riguarda il supporto sia per elaborazioni batch sia per elaborazioni streaming. L'analisi offline di un video richiede spesso la disponibilità dell'intera sequenza, mentre una pipeline progressiva richiede di processare i dati man mano che vengono acquisiti. SEF deve quindi poter gestire pipeline ibride, nelle quali alcuni componenti lavorano in streaming e altri richiedono la materializzazione dei dati.

Un quarto vincolo riguarda l'osservabilità. In un sistema modulare, l'utente deve poter comprendere non solo il risultato finale, ma anche il modo in cui la pipeline è stata costruita ed eseguita. Per questo motivo, SEF introduce meccanismi come il piano di esecuzione, gli eventi, il monitoraggio delle run e una gestione tipizzata degli errori.

Infine, un vincolo significativo riguarda la portabilità degli output. I risultati prodotti da una pipeline non devono essere legati a una specifica interfaccia grafica. Un visualizzatore non dovrebbe restituire direttamente un widget Streamlit o un oggetto specifico di un notebook, ma un artefatto astratto che possa essere successivamente mostrato, salvato o trasformato da adapter differenti.

Tabella 2.4: Sintesi delle principali scelte progettuali derivate dai requisiti

Problema	Scelta adottata	Tradeoff
Evitare dipendenze dirette da librerie esterne nel core	Separazione tra core, built-in e adapter	Maggiore pulizia architetturale, ma più livelli di astrazione.
Supportare API, CLI e UI senza duplicazione	Modello comune di pipeline e configurazione	Maggiore coerenza, ma schema configurativo più generale.
Gestire sia batch sia streaming	Runtime basato sulle capability degli stage	Maggiore flessibilità, ma runtime più complesso.
Rendere spiegabile l'esecuzione	Execution plan, eventi e monitor	Migliore debugging, ma necessità di mantenere allineati planner e runtime.
Supportare plugin personalizzati	Contratti, registry e decorator	Estensione più semplice, ma maggiore complessità nel sistema di risoluzione.
Mantenere gli output indipendenti dalla UI	Artifact system indipendente dall'interfaccia utente	Maggiore portabilità, ma necessità di adapter di rendering.
Gestire errori in modo robusto	Errori tipizzati con contesto	Diagnostica più chiara, ma maggiore lavoro di modellazione.

Queste scelte mostrano come i requisiti individuati non siano rimasti semplici dichiarazioni astratte, ma abbiano influenzato direttamente la struttura del framework. L'obiettivo non è stato soltanto costruire una soluzione funzionante per un singolo caso d'uso, ma definire una base architetturale capace di sostenere evoluzioni future.

2.8 Casi d'uso di riferimento

Durante lo sviluppo sono stati considerati diversi scenari applicativi utili a validare la generalità del framework. Il primo caso sperimentale, come già evidenziato, è stato il tracking di oggetti in movimento tramite OpenCV. Tale scenario ha consentito di verificare la possibilità di acquisire frame, applicare algoritmi di tracciamento, produrre risultati intermedi e generare output visuali.

Un secondo scenario di interesse è rappresentato dalla *motion magnification*, collegata all'analisi di micro-movimenti. In questo caso, il video viene trattato come sorgente di segnali deboli, che devono essere amplificati e analizzati per evidenziare variazioni difficilmente osservabili direttamente.

Altri scenari considerati includono il dense optical flow, il rilevamento di marker ArUco per l'analisi di spostamenti, la stima della posa umana tramite modelli di *pose estimation* e il conteggio di attraversamenti rispetto a una linea virtuale. Questi casi d'uso non sono equivalenti tra loro: alcuni richiedono elaborazione frame-by-frame, altri analisi di sequenze complete, altri ancora produzione progressiva dei risultati.

Proprio questa varietà ha rafforzato la necessità di progettare un framework capace di supportare componenti diversi all'interno di una struttura comune.

Tabella 2.5: Casi d'uso di riferimento per la definizione dei requisiti

Caso d'uso	Obiettivo	Implicazioni sui requisiti
Tracking di oggetti	Seguire la posizione di uno o più oggetti nel tempo.	Necessità di stage sostituibili e output visuali.
Motion magnification	Evidenziare micro-movimenti non percepibili a occhio nudo.	Supporto a pipeline di analisi video più articolate.
Dense optical flow	Stimare il movimento apparente tra frame consecutivi.	Elaborazione frame-by-frame e gestione di dati densi.
Marker ArUco	Estrarre spostamenti o moto relativo da marker visuali.	Signal extraction e analisi di serie temporali.
Pose estimation	Stimare la posa umana e ricavare misure o posture.	Integrazione di modelli esterni tramite adapter.
Barrier counting	Contare attraversamenti rispetto a una soglia o linea virtuale.	Eventi, analisi e output interpretabili.

La presenza di casi d'uso così diversi mostra perché SEF non potesse essere progettato come semplice applicazione verticale. Una soluzione limitata a un solo scenario avrebbe probabilmente semplificato l'implementazione iniziale, ma avrebbe ridotto in modo significativo la riusabilità del sistema.

2.9 Sintesi del capitolo

In questo capitolo sono stati descritti il contesto, le motivazioni e i requisiti che hanno portato alla progettazione di SEF. Il framework nasce dall'esigenza di superare i limiti di soluzioni monolitiche e difficilmente riutilizzabili, proponendo una struttura modulare per l'analisi e l'elaborazione di flussi video.

Il problema centrale non riguarda soltanto l'esecuzione di algoritmi di *computer vision*, ma l'organizzazione dell'intero processo di trasformazione del video: dall'acquisizione dei frame fino alla produzione di segnali, risultati analitici e artefatti visualizzabili. Per soddisfare questo obiettivo sono stati definiti requisiti funzionali legati a pipeline modulari, configurazione dichiarativa, plugin, modalità batch e streaming, CLI, interfaccia grafica e gestione degli output.

Sono stati inoltre individuati requisiti non funzionali come modularità, basso accoppiamento, estendibilità, osservabilità, riproducibilità, usabilità e controllo delle risorse. Tali requisiti costituiscono la base delle scelte architetturali descritte nel capitolo successivo, nel quale verrà analizzata l'organizzazione interna del framework e il modo in cui SEF traduce questi obiettivi in componenti software concreti.

3. Architettura del framework

In questo capitolo viene descritta l'architettura di SEF e il modo in cui i requisiti introdotti nel capitolo precedente sono stati tradotti in componenti software concreti. Dopo una panoramica del modello generale della pipeline vengono presentati il core del framework, i principali contratti architetturali, il runtime, i meccanismi di estensione e le implementazioni di riferimento.

3.1 Architettura interna del framework

3.1.1 Flusso generale della pipeline

La pipeline segue tutti i passaggi dall'estrazione di frame del video fino all'emissione di dati utili all'utente finale. Per far ciò la pipeline è stata divisa nelle seguenti componenti principali:

Frame extractor: Componente che traduce il video in uno specifico formato in una sequenza di *Frame*, ossia oggetti dell'omonima classe che soddisfano determinate condizioni e che possono essere manovrati in maniera generica per le qualsivoglia operazioni su di essi. Varie implementazioni di questo componente possono adattarsi ai più disparati formati video (.mp4, .mov ecc), ma anche a situazioni strutturalmente diverse, come la lettura di un video in "real-time" (magari da una telecamera attiva 24/7) o di file molto grandi contenuti in database esterni (nel cui caso si adotterebbe un approccio *bufferizzato*, come vedremo in seguito).

Frame processor: Si occupa della manipolazione dei *Frames* ottenuti nello stage precedente. In generale le operazioni possono limitarsi a semplici trasformazioni su singoli frame (come un ridimensionamento del frame, o una manipolazione sulla scala di colori usata) oppure a modifiche che riguardano più frame (è questo il caso di molte operazioni che richiedono la conoscenza di uno "stato pregresso" del video, in modo da effettuare delle modifiche in modo coerente ad esso, come avviene per la rimozione di elementi non importanti nel video o la magnificazione dei movimenti). Vale la pena notare che il processing va ad agire in maniera totalmente generica e disaccoppiata

rispetto al formato o al tipo originale di video, grazie all'astrazione rappresentata dai *Frames*.

Signal extractor: Ha lo scopo di trasformare i *Frames* ottenuti e ripuliti in precedenza in veri e propri dati. La tipologia di questi ultimi dipende dalle applicazioni del caso considerato, anche se in genere la maggior parte delle volte si tratta di considerare qualche grandezza numerica, come posizione di particolari elementi nel video, numero di oggetti che eseguono una determinata operazione, distanze relative tra punti ben precisi tra i frame, e via dicendo. Anche in questo caso i prodotti ottenuti sono "standardizzati" con la classe *Signal*, che non è altro che una lista di *SignalSample*.

Signal cleaner: L'obiettivo è simile a quello prefissato per i *Frame processor*, ma applicato ai *Signal*; i dati vengono quindi ripuliti tramite diverse operazioni prima di essere analizzati nella successiva fase. Come specificato prima, abbiamo in genere a che fare con i numeri, quindi le operazioni su questi dati saranno molto spesso operazioni vettoriali e matriciali numeriche, anche se chiaramente possiamo pensare a delle possibili eccezioni.

Analyzer: È il cuore dell'analisi del progetto, come suggerisce il nome, analizza i *Signal* prodotti in precedenza per produrre un altro tipo di dato standardizzato classificato con il nome *Data*. Di primo acchito potrebbe sembrare un processo ridondante con i due precedenti, essendo che spesso si ottengono dati numerici simili; tuttavia si è voluto distinguere la pipeline in queste due parti poiché i *Signal* rispetto ai *Data* non contengono informazione semantica. La posizione di una macchina in strada che si muove non contiene nessun significato in sé, e così com'è risulta grezza e poco usabile. Analizzandola però possiamo giungere a conclusioni più interessanti, ottenendo ad esempio la sua velocità nel tempo, da cui si può considerare se la macchina stia ad esempio superando i limiti di velocità predisposti oppure no, e altri dati simili. Diversi *Data* possono essere costruiti da un singolo *Signal*, e per questo, anche nell'architettura della nostra pipeline abbiamo implementato la possibilità di ottenere diversi risultati usando combinazioni di diversi *Analyzer*.

Visualizer: Rappresenta la parte conclusiva della pipeline costruita. Essenzialmente il suo scopo è quello di prendere gli oggetti *Data* ottenuti ed elaborarli in modo tale da poterli presentare visualmente all'utente finale. Chiaramente la prima idea per spiegare meglio dei dati numerici è quella di "graficarli": in tal senso la maggior parte dei *Visualizer* implementati è costituiti da generatori di grafici, istogrammi, diagrammi, o comunque qualsiasi tipo di immagine che renda l'informazione ottenuta in forma visuale. Comunque, la struttura flessibile di questo componente non vieta in alcun modo la produzione di altri tipi di risultati, ed è per questo che l'output finale è codificato attraverso la classe standardizzata dei *Visual artifacts*.

La struttura riportata è la base concettuale dell'architettura, anche se essa si concretizza nella realtà con strumenti ed accorgimenti più complessi, nati dall'esigenza di soddisfare un'ampia gamma di problemi che si potevano presentare nel contesto dell'elaborazione video; incontreremo più avanti infatti come è stata fornita anche la possibilità di ottenere risultati intermedi durante l'esecuzione di questo processo, oppure come dividere questa catena lineare con "branching" più generali, e altro ancora.

3.1.2 Separazione tra core e adattatori

Il progetto è stato strutturato seguendo una ben precisa stratificazione, ispirandoci alle linee guida della *Clean Architecture* (vedi [Mar17]). La Clean Architecture è efficace perché separa la logica centrale dell'applicazione dai dettagli tecnici esterni. In questo modo interfaccia grafica, database, framework o strumenti di automazione possono cambiare senza compromettere il nucleo del progetto. Ne risulta un software più modulare, testabile, manutenibile e facilmente estendibile nel tempo.

Come si può vedere dalla figura 3.1, il cuore del nostro framework è rappresentato dal *core*, esso presenta tutti i concetti, rigorosamente astratti, che servono da base per gli sviluppi concreti.

Troviamo chiaramente tutte le interfacce usate, che fungono da "modelli" per la produzione di ogni componente della pipeline citato prima. Chiunque voglia implementare un proprio strumento deve seguire i contratti riportati dalle interfacce, in modo da poter seguire le regole per l'interazione con tutte le altre componenti.

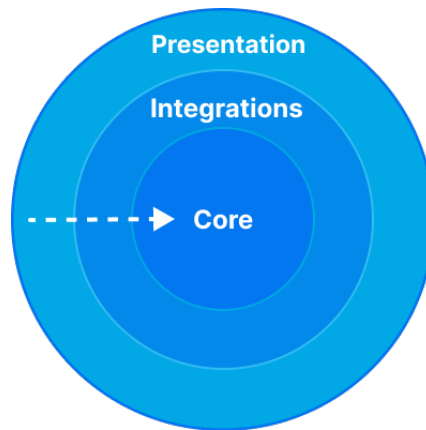


Figura 3.1: Regola delle dipendenze nella Clean Architecture.

Troviamo anche tutti gli *artefatti* citati nella sezione 3.1.1. Essi sono parte del core perché rappresentano le informazioni standardizzate che si scambiano i componenti; anche le interfacce li usano, per evidenziare e chiarire nei contratti dei componenti come sono fatti i dati che ricevono e in che formato devono essere restituiti.

Anche le classi predisposte per la formazione della pipeline sono nel core, perché rappresentano la forma dello "scheletro" della catena di elaborazione video del framework. Definiscono l'ambiente fondamentale entro cui operano gli strumenti già citati.

Sono presenti anche altre classi che hanno comunque il ruolo di definire la struttura sottostante del progetto, come per la parte di *eventi* e *plugin*. Vedremo meglio in seguito di cosa trattano.

Nel layer intermedio si trovano le *integration*: qui le componenti astratte definite concettualmente nel core vengono alla luce sotto forma delle più disparate implementazioni. In genere, per le integrazioni che abbiamo fornito di default, troviamo principalmente classi realizzate con la libreria *OpenCV*, basata su computer vision e image-processing; anche se altre implementazioni sono state realizzate con *YOLO*, *Matplotlib* o altri algoritmi che non hanno libreria (come ad esempio quello per il moto magnificato).

È bene chiarire che tutte queste implementazioni, per via del fatto che rispettano tutti i contratti del core, sono intercambiabili e si prestano a molte combinazioni, cosa che favorisce un uso più generale, personalizzabile e più mirato ad ogni problema che si presenti.

Infine, nel layer più esterno troviamo la *presentation*, cioè l'insieme dei punti di accesso attraverso cui l'utente interagisce con il framework senza dover conoscere direttamente i dettagli interni del core. In questo progetto tale livello è rappresentato principalmente dalla *CLI*, dall'interfaccia grafica *SEF Studio*, realizzata con Streamlit, e dalle API pubbliche esposte dal package.

La *presentation* non contiene la logica fondamentale di elaborazione video, ma si occupa di tradurre le azioni dell'utente in configurazioni, comandi o chiamate verso i servizi applicativi sottostanti. Ad esempio, l'interfaccia grafica permette di comporre visualmente una pipeline, selezionare componenti da un registro predisposto, modificare i parametri degli stadi e consultare gli output prodotti. Allo stesso modo, la CLI permette di validare configurazioni, ispezionare i componenti disponibili ed eseguire pipeline passando per il terminale piuttosto che per un'interfaccia di altro tipo.

Questo livello dipende quindi dagli strati più interni, ma non avviene il contrario: il core non conosce Streamlit, la CLI o altre forme di presentazione. Tale scelta consente di sostituire o aggiungere nuove interfacce, ad esempio una dashboard web diversa, un notebook o un servizio remoto, mantenendo invariata la logica centrale del framework. In questo modo la separazione tra core, integrazioni e presentation rende l'architettura più flessibile, perché ogni livello ha una responsabilità precisa e può evolvere senza compromettere gli altri.

3.1.3 Contratti e astrazioni principali

Il progetto SEF definisce una pipeline a stadi tipizzati, nella quale ogni componente comunica con il successivo tramite contratti espliciti basati su interfacce e artefatti. Il contratto generale è che ogni stadio riceva un tipo di dato ben definito, produca il tipo atteso dallo stadio successivo e preservi, quando possibile, ordine temporale, indici di frame, timestamp e metadati.

Frame extractor: L'interfaccia che lo definisce è *IFrameExtractor*. Essa contiene il metodo *extract()*, che è predisposto per restituire un *FrameBuffer* (contenente una serie di *Frame*). Il meccanismo implicito di questo componente è che il video venga analizzato e spezzettato in *Frame*, posti in un buffer, in modo che l'estrazione si disaccoppi dall'elaborazione, fornendo velocità e parallelismo al sistema). Il buffer restituito deve essere chiuso al termine dell'estrazione, in modo da evitare errori. Nel caso di estrazione streaming, la pubblicazione di *Frame* nel buffer (bounded) è vincolata a rispettare delle politiche di latenza determinate in precedenza.

Frame processor: I *Frame Processor* sono divisi in due contratti. Un *ISingleFrameProcessor* implementa il metodo *process(frame)*, volto alla trasformazione di un singolo *Frame* alla volta; è questo il caso di operazioni stateless che non richiedono la conoscenza di un "intorno" di frame (come ad esempio rendere il frame in "scala di grigi"). Nell'altro scenario, un *IFrameBufferProcessor* implementa il metodo *process(buffer)* e riceve, potenzialmente, l'intera sequenza di frame. Il *Frame Processor* considerato decide in modo autonomo quanti frame aspettare prima di generare in uscita un altro frame o prima di consumare dal buffer di input un frame; Il contratto implicito è che tale processor gestisca quest'operazione in modo efficiente, attendendo solo una finestra di frame utile per la trasformazione richiesta, senza sovraccaricare la memoria. In entrambi i casi il risultato deve essere un *Frame* o un *FrameBuffer* valido per gli stadi successivi, preservando, in genere, l'ordine (a meno che la trasformazione del video vada a modificare proprio quest'aspetto).

Frame exporter: Il *Frame exporter* è un componente aggiuntivo della pipeline per accedere a risultati intermedi dell'elaborazione, senza fermarla. Questo componente è rappresentato dall'interfaccia *IFrameExporter*. Essa implementa il metodo *export(buffer, context)*; il suo scopo principale è quello della produzione di artefatti visuali "al volo", in modo da non interrompere la pipeline. Tale metodo deve restituire un *FrameExportResult* contenente sia gli artefatti generati sia un *FrameBuffer* ancora consumabile dal successivo *SignalExtractor*.

Signal extractor: Qui i contratti sono definiti dall'interfaccia *ISignalExtractor*. Essa implementa il metodo *extract(buffer)* e converte i frame processati in precedenza in un

ISignal (interfaccia per i *Signal*). Il *Signal* prodotto deve essere composto da campioni *ISignalSample*. Ogni campione mantiene il riferimento temporale tramite i campi *frame_index*, *timestamp_seconds* (opzionale) e *metadata*. La variante streaming pubblica campioni progressivamente tramite *extract_into()* in modo analogo alla variante streaming del *IFrameExtractor*.

Signal cleaner: Un *ISignalCleaner*, interfaccia del *Signal cleaner*, implementa il metodo *clean(signal)*, che trasforma un segnale già estratto, ad esempio filtrandolo, normalizzandolo o correggendolo. Il contratto implicito è mantenere la semantica del segnale, l'ordine dei campioni e i metadati quando possibile. La variante streaming implementa *clean_into()*, con logica streaming analoga alle precedenti.

Analyzer: L'interfaccia *IAnalyzer* implementa da contratto il metodo *analyze(signal)*; esso converte un *ISignal* in un risultato *IData* contenente significato (rispetto al precedente). Questo stadio separa l'elaborazione numerica dalla visualizzazione. Gli analyzer streaming possono pubblicare risultati progressivi, attraverso le componenti *live analyzer*, che producono "al volo" Data intermedi pronti per essere visualizzati. Anche in questo caso devono comunque restituire un risultato finale autorevole.

Visualizer: Un *IVisualizer* implementa il metodo *render(data, context)* che si occupa di convertire un *IData* in una tupla di *VisualArtifact*. Il contratto richiede che gli artefatti siano indipendenti dall'interfaccia grafica, per ovvi motivi: non devono dipendere da Streamlit, finestre OpenCV, notebook o stato del browser. L'adattamento alla UI è responsabilità di componenti esterni.

Artefatti principali: Citati anche in precedenza, qui raccogliamo un insieme di contratti degli artefatti principali utilizzati in questa catena di elaborazione. Un *Frame*, dato utile dello stage per l'estrazione video e l'elaborazione del segnale, contiene i campi *image*, *index*, *timestamp_seconds* e *metadata*; l'immagine non viene copiata automaticamente, quindi la mutabilità dei pixel resta responsabilità dei componenti. Un *FrameBuffer* è un buffer iterabile (per poterlo scorrere in maniera agevole), thread-safe e con proprietà di ordinatezza (in modo da evitare di invertire i Frame). Può essere chiuso a piacimento se la situazione lo richiede. Un *ISignal* è una collezione ordinata di *ISignalSample*, questi ultimi invece sono campioni di segnale che contengono le informazioni di base dell'analisi del Signal extractor effettuata; contiene campi come *timestamp_seconds*, *metadata* ecc. Un *IData* è il tipo base dei risultati analitici, che rimane spoglio di campi o contratti particolari per lasciare più libertà nelle implementazioni. Un *VisualArtifact* descrive un output UI-agnostico tramite i campi *artifact_id*, *kind*, *role*, *title*, *description* e *metadata*; le specializzazioni includono immagini, video, file video, video deferred, tabelle, JSON e testo.

Pipeline context e output: Il *PipelineContext* è il contratto di composizione della pipeline: richiede come componenti necessari un *frame_extractor*, un *signal_extractor* e almeno un *analyzer*; il motivo è chiaro: sono i componenti minimi che realizzano un'elaborazione; gli altri stadi sono opzionali. Esso inoltre normalizza le collezioni a tuple, rifiuta valori *None* e fornisce ulteriori controlli per una corretta esecuzione del programma. Il risultato finale è un oggetto del tipo *PipelineOutputs*, che separa i valori ottenuti nei campi *results*, *final_artifacts*, *debug_artifacts*, *metadata* e *intermediate_frames*.

Contratti trasversali: Esistono contratti trasversali disponibili per tutte le componenti della pipeline, come ad esempio quello di *StageCapabilities*. Ogni componente può dichiarare *StageCapabilities*, specificando se supporta streaming, se richiede la sequenza completa, se è stateful, se preserva l'ordine, se supporta parallelismo sui frame e se è adatto al realtime. Nel runtime streaming i buffer devono essere chiusi a fine produzione, abortiti in caso di errore non recuperabile e non devono ricevere nuovi elementi dopo chiusura o abort. Il sistema assume inoltre che gli errori pubblici siano tipizzati e che eventi, branching, e simili interagiscano tramite interfacce dedicate, evitando dipendenze dirette da implementazioni concrete.

3.1.4 Plugin Registry e configurazione versionata

Il *PluginRegistry* rappresenta il meccanismo centrale con cui la pipeline risolve dinamicamente i componenti disponibili. Ogni plugin viene identificato tramite una categoria funzionale, un nome canonico, eventuali alias e un insieme di metadata descrittivi. In questo modo, la costruzione della pipeline non dipende direttamente dalle classi concrete, ma da identificatori stabili dichiarati nella configurazione. Il registry consente quindi di separare la descrizione della pipeline dalla sua implementazione, facilitando estensione, manutenzione e sostituzione dei componenti.

Le categorie sono modellate tramite la classe *PluginCategory* e riflettono le principali responsabilità e caratteristiche del sistema: *frame_extractor* per l'acquisizione dei frame, *single_frame_processor* per elaborazioni locali sul singolo frame, *frame_buffer_processor* per operazioni su sequenze temporali di frame, *signal_extractor* per estrarre segnali dai dati visivi, *signal_cleaner* per la pulizia dei segnali, *analyzer* per l'analisi dei risultati, *visualizer* per la produzione di output grafici e *branching_rule* per la gestione di diramazioni condizionali nella pipeline.

Ogni componente registrato è descritto da una *PluginDefinition*. Tale classe include la categoria, il nome canonico, la *factory* responsabile dell'istanziamento, una descrizione, la versione del plugin, eventuali alias, metadata aggiuntivi e il campo *factory_path*, utile per riferire in modo esplicito il punto di costruzione del componente. Il registry si pone come raccolta non statica bensì dinamica di elementi, esponendo operazioni per la registrazione di plugin tramite *register* e *register_definition*, recuperabili con *get*, e istanziabili con *create*; sono inoltre disponibili altri metodi come *list*, *contains*, *availa-*

ble_names, *categories*, con operazioni deducibili facilmente dal loro nome. È possibile anche produrre uno *snapshot* e generare descrizioni leggibili con *describe*.

Un aspetto importante è la robustezza del meccanismo di registrazione. Gli identificatori vengono validati per evitare nomi ambigui o non conformi; gli alias permettono di mantenere retrocompatibilità quando un componente cambia nome; gli snapshot immutabili offrono una vista stabile dello stato del registry; l'uso di lock garantisce la thread safety durante registrazione, consultazione e creazione dei plugin. Nel progetto, questo meccanismo collega in modo uniforme i componenti.

La configurazione della pipeline è resa riproducibile tramite un sistema di versioning esplicito. Il campo *schema_version*, consente di dichiarare quale versione dello schema viene utilizzata. La classe *VersionedPipelineConfig* incapsula questa informazione, mentre *PipelineConfigVersionManager* e *PipelineConfigMigration* permettono di gestire l'evoluzione dello schema nel tempo. La funzione *normalize_pipeline_config* uniforma le configurazioni in ingresso, rendendole compatibili con il formato atteso dal builder.

Infine, *ConfigPipelineBuilder* conserva in *source_config* una copia compatta della configurazione originale, utile per export, debugging e riproducibilità sperimentale. Questa scelta permette di ricostruire non solo la pipeline eseguita, ma anche il contesto dichiarativo da cui essa deriva. A supporto di questo processo intervengono *PipelineConfigExporter*, *PipelineCodeExporter* e *PipelineExportUtils*, che permettono l'esportazione in JSON, YAML e codice Python ricostruibile; una soluzione che accentua ancor di più le caratteristiche di usabilità del framework.

3.1.5 Pianificazione dell'esecuzione e runtime ibrido

Il framework non esegue la pipeline in modo diretto e ingenuo, ma costruisce un piano di esecuzione, in modo da coordinare al meglio le operazioni. Questo passaggio è affidato alla classe *PipelineExecutionPlanner*. Esso è un componente *read-only* non modificabile, che ispeziona il *PipelineContext*, le capacità dichiarate dagli stage e le policy di esecuzione prima dell'avvio effettivo. L'obiettivo è determinare, per ciascuna porzione della pipeline, se sia più opportuno operare in modalità batch, streaming o ibrida, evitando assunzioni globali troppo rigide. In questo modo otteniamo flessibilità e al tempo stesso efficienza.

Il risultato della pianificazione è rappresentato dal piano *PipelineExecutionPlan*, composto da una sequenza di *ExecutionPlanStage*. Ogni stage del piano contiene informazioni come lo *stage id*, lo *stage group*, il nome del componente, la modalità di esecuzione scelta, le capacità dello stage, e altre caratteristiche che articolano nel dettaglio la pianificazione del lavoro. È possibile andare a definire meglio anche un eventuale stima del consumo di memoria, per fini di coordinazione. Con questa pianificazione, il *PipelineExecutionPlan* non descrive solo l'ordine degli stage, ma anche il modo in cui i dati devono fluire tra essi. Le decisioni del planner sono guidate da *DefaultPipelineExecutionPolicy*, che stabilisce quando aprire, preservare o chiudere segmenti streaming. La

policy considera le capacità del componente corrente, la domanda degli stage successivi, la presenza di consumatori progressivi e le stime di costo. Per esempio, uno stage che produce dati in streaming può mantenere questa modalità se gli stage downstream sono in grado di consumarli progressivamente; al contrario, se uno stage successivo richiede l'intero insieme dei dati, il planner introduce un boundary conseguente a tale necessità.

Le stime sono raccolte in *PipelineExecutionEstimates*, che valuta il costo approssimativo delle code di frame, signal e data, oltre alla possibile materializzazione completa dei frame. A supporto di queste decisioni interviene *PipelineExecutionLookahead*, usato per analizzare gli stage successivi e capire se essi possano continuare a consumare dati in streaming oppure richiedano una conversione verso una rappresentazione batch.

A runtime, l'esecuzione è organizzata in segmenti specializzati a seconda dei vari componenti già incontrati nella pipeline. *FrameSegmentExecutor* gestisce estrazione, processamento ed esportazione dei frame; *SignalSegmentExecutor* si occupa di estrazione e pulizia dei segnali; *AnalysisSegmentExecutor* coordina analyzer e visualizer. Questa separazione consente di applicare strategie diverse e mirate a seconda del tipo di dato e della fase della pipeline, mantenendo il controllo sul flusso complessivo.

Quando una porzione streaming deve essere consegnata a uno stage batch, interviene *PipelineBoundaryMaterializer*. Questo componente materializza frame o signal, trasformando un flusso progressivo in una struttura completa disponibile in memoria o in una rappresentazione intermedia. In alcuni casi, può anche ripubblicare un signal già materializzato come stream, permettendo a componenti successivi di tornare a una modalità progressiva, con conseguente aumento dell'efficienza.

Il risultato è un runtime ibrido: una stessa pipeline può contenere tratti streaming, tratti batch e visualizer progressivi senza imporre una singola modalità globale. Questa scelta rende il framework più flessibile, perché consente di sfruttare lo streaming dove riduce memoria e latenza, ma mantiene la compatibilità con componenti che richiedono una vista completa dei dati.

3.1.6 Buffer, latenza e supporto al realtime

Il supporto al realtime richiede un controllo esplicito dei buffer e della latenza; infatti, le dimensioni e le caratteristiche dei buffer vanno sicuramente ad incidere su tutta l'esecuzione, sulla sua velocità e sulle sue prestazioni. Nel framework questo comportamento è configurato tramite *StreamRuntimeConfig*, che definisce le dimensioni dei buffer per i diversi tipi di dato: *frame_buffer_size*, *signal_buffer_size* e *data_buffer_size*. La stessa configurazione include anche *latency_policy*, cioè la strategia da applicare nel caso in cui la produzione dei dati fosse più veloce del loro consumo. Questo introduce un compromesso centrale tra completezza dei dati, uso della memoria e reattività del sistema (vedremo poco più avanti le soluzioni proposte nel progetto).

Per i frame viene utilizzato *FrameBuffer*, una coda bounded e thread-safe con capacità pubblica. Essendo bounded, il buffer non può crescere indefinitamente: quando

si riempie, il comportamento dipende dalla policy di latenza selezionata. Il buffer supporta metodi come *try_put*, *drop_oldest*, *fill_ratio*, *close* e *abort*, che implementano operazioni analoghe ai loro nomi. Queste primitive permettono sia una chiusura ordinata dello stream (*close*) sia un'interruzione anticipata in caso di errore o cancellazione (*abort*). *SignalBuffer* e *DataBuffer* estendono questa logica a dati che possono essere consumati da più componenti; questa necessità si presenta poichè i vari *Signal* prodotti in un buffer possono essere "consumati" da più Analyzer. Discorso analogo per il buffer di *Data*. Essi supportano un modello multi-consumer tramite *set_consumer_count* e *subscribe*, la cui logica si basa essenzialmente sul fatto che i vari componenti possono "isciversi" a più buffer, il quale a sua volta gestisce tutto il resto. In questo modo lo stesso segnale o dato intermedio può essere distribuito verso più analyzer o visualizer senza duplicare inutilmente la logica di produzione. Il campione viene considerato rilasciabile solo quando tutti i consumer previsti lo hanno consumato.

Le policy di latenza determinano il comportamento del sistema quando il buffer è saturo. La *BlockingFrameLatencyPolicy* privilegia completezza e riproducibilità: il produttore attende finché non c'è spazio disponibile, evitando perdita di frame. *DropNewestFrameLatencyPolicy* scarta invece il nuovo frame quando il buffer è pieno, preservando i dati già accodati. *DropOldestFrameLatencyPolicy* elimina il frame più vecchio per fare spazio a quello appena prodotto, favorendo una visione più aggiornata dello stream. Infine, *AdaptiveSamplingFrameLatencyPolicy* modifica dinamicamente l'intervallo di campionamento in base alla pressione sul buffer, riducendo la quantità di frame immessi quando il sistema non riesce a consumarli abbastanza velocemente.

Queste strategie sono collegate alle metriche runtime, che registrano frame accettati, frame scartati, frame visti e, nel caso dell'adaptive sampling, l'intervallo corrente di campionamento. Tali metriche rendono osservabile il comportamento del sistema e permettono di valutare se la pipeline stia privilegiando la completezza dei dati oppure la riduzione della latenza.

Il compromesso principale è quindi tra modalità offline e realtime. La policy *blocking* è adatta quando è importante conservare tutti i frame, per esempio in analisi riproducibili dove la precisione è d'obbligo. Le strategie *drop_oldest*, *drop_newest* e *adaptive_sampling* sono invece più adatte a scenari realtime, dove un backlog eccessivo renderebbe la pipeline poco reattiva. In questi casi è preferibile perdere parte dell'informazione pur mantenendo una latenza contenuta.

3.1.7 Eventi, branching e controllo real-time

Non abbiamo ancora trattato un importante, possibile scenario della nostra pipeline; immaginiamo di voler analizzare un particolare video, e di notare, nel momento dell'analisi dei dati raccolti, che in esso vi è un altro elemento di interesse che merita di una seconda analisi, diversa e più mirata della precedente. Un sistema più semplicistico prevederebbe di effettuare una seconda analisi da zero, sul dettaglio specifico, distac-

candosi dalla prima, ma ciò ci farebbe perdere tutto il contesto dell'analisi precedente (a meno che non lo si voglia riportare a mano in componenti specifici) e risulterebbe nel complesso un'operazione comunque più tediosa.

Il sistema di *Branching* della pipeline è pensato proprio per ovviare tale problema, fornendo un sistema ad eventi che permette di effettuare più analisi ramificate nella stessa esecuzione, coordinando i vari rami tramite eventuali condizioni per l'attivazione.

Il framework include un sistema a eventi che permette di separare l'esecuzione principale della pipeline da reazioni secondarie, notifiche e meccanismi di estensione. La struttura di base è composta da *Event* ed *EventBus*. Gli eventi vengono pubblicati in modo sincrono tramite *dispatch*, mentre *publish* fornisce un wrapper asincrono. L'event bus supporta subscriber con wildcard, isola gli handler tra loro e registra eventuali errori tramite logging, evitando che un handler fallito comprometta l'intero flusso di esecuzione.

Tra gli eventi più importanti vi sono i lifecycle events definiti in *PipelineLifecycleEvent*. Questi eventi descrivono le fasi principali del ciclo di vita di una pipeline e permettono a servizi esterni, monitor o interfacce utente di reagire in modo uniforme allo stato dell'esecuzione.

Un caso specifico è rappresentato da *PipelineEvent*, che modella l'evento *pipeline.trigger*. Esso contiene il *pipeline_id*, il *PipelineContext* e i metadata di esecuzione necessari per avviare una pipeline secondaria a partire da un evento. Questo consente di trasformare eventi di dominio in nuove esecuzioni, mantenendo esplicito il contesto e rendendo tracciabile la relazione tra pipeline principale e pipeline secondarie.

Gli extractor possono partecipare a questo meccanismo tramite l'interfaccia *IEventEmitter*, emettendo eventi significativi durante l'elaborazione. Un esempio, tratto dal lato implementativo, è contenuto in *OpenCVMultiObjectSignalExtractor*, che produce eventi come *track_created* e *track_lost*. Tali eventi descrivono cambiamenti rilevanti nel dominio applicativo, ad esempio la comparsa o la perdita di una traccia, e possono diventare il punto di partenza per ulteriori elaborazioni. L'utente finale che utilizza tali strumenti ha comunque la possibilità di sfruttare il sistema ad eventi in modo personalizzato al fine di soddisfare le particolari richieste della sua specifica elaborazione.

Il branching è coordinato da *BranchingCoordinator*, che ascolta gli eventi di dominio, applica una o più regole definite da *IBranchingRule*, costruisce un *PipelineContext* secondario e dispatcha un *PipelineEvent*. In questo modo, l'estendibilità non richiede di modificare la pipeline principale: nuove regole possono essere aggiunte per reagire a eventi specifici e attivare flussi specializzati.

Oltre al branching, il framework espone diversi strumenti di osservabilità runtime. *ThreadedPipelineRunner* gestisce l'esecuzione asincrona delle pipeline, mentre *InMemoryPipelineMonitor* mantiene lo stato corrente e storico delle esecuzioni. Le informazioni sono rappresentate tramite *PipelineRunSnapshot*, che descrive stati come *queued*, *running*, *succeeded*, *failed* e *cancelled*. Gli output prodotti vengono invece gestiti tramite

InMemoryPipelineOutputStore, così da poter essere recuperati, visualizzati o esportati dopo l'esecuzione.

3.2 Dettagli implementativi

Ora, sempre con la Clean Architecture ([Mar17]) in mente, ci apprestiamo alla descrizione dello strato più esterno del framework, ossia l'implementazione dei vari componenti. Vedremo come i componenti della pipeline introdotti in precedenza possono essere utilizzati come strumenti pratici, come si configurano e come si relazionano l'uno con l'altro attraverso la struttura organizzativa della pipeline.

3.2.1 Estrazione e processing dei frame

L'estrazione dei frame al momento è basata unicamente su due classi: *OpenCVBufferedFrameExtractor* e *OpenCVWebcamFrameExtractor*. Con la prima è possibile ottenere una sequenza ordinata di frame a partire da un video salvato in memoria; per far ciò la classe fa affidamento alla libreria *OpenCV*. È possibile configurare il meccanismo di estrazione con vari parametri aggiuntivi, come *stride* (che ci permette di selezionare e propagare un frame ogni certo numero di estrazioni), *max_frames* (che limita il massimo numero di frame) ecc. In questa fase i frame possono essere arricchiti da timestamp, e altri tipi di metadata. Con *OpenCVWebcamFrameExtractor* invece, possiamo acquisire direttamente frame da una videocamera in tempo reale; anche qui abbiamo parametri simili ai precedenti, per personalizzare l'acquisizione. Chiaramente, essendo un'estrazione real-time, il buffer in output potrà adottare politiche di gestione frame più mirate all'efficienza sul riempimento del buffer più che sulla conservazione di ogni immagine.

Esistono diversi processori di frame, molti dei quali operano su singoli frame con trasformazioni che implementano algoritmi di *OpenCV*. Tra i processor disponibili sono presenti componenti dedicati a trasformazioni geometriche (ridimensionamento, rotazione e zoom), conversioni cromatiche (scala di grigi) e tecniche di miglioramento dell'immagine, come l'equalizzazione dell'istogramma.

Tra le implementazioni più significative rientra *DynamicObjectRemovalFrameProcessor*, questo implementa una tecnica batch di background subtraction basata sulla mediana temporale. Lo sfondo statico viene stimato calcolando, per ogni posizione spaziale e canale cromatico, la mediana di un sottoinsieme di frame di lunghezza configurabile distribuiti lungo la sequenza. Per ciascun frame viene quindi calcolata la differenza assoluta rispetto allo sfondo stimato. La differenza viene ridotta a una rappresentazione monocanale e sottoposta a soglia, producendo una maschera binaria delle regioni considerate dinamiche. La maschera è successivamente raffinata mediante apertura, chiusura e dilatazione morfologica, seguite dall'eliminazione delle componenti connesse

di area inferiore a una soglia configurabile. Dopo l'eventuale esclusione delle regioni protette, i pixel appartenenti alla maschera vengono sostituiti con i corrispondenti valori dello sfondo temporale. Il metodo è deterministico ed efficace in presenza di telecamera statica e oggetti transitori, ma risulta sensibile a variazioni di illuminazione, movimenti della camera e oggetti che permangono a lungo nella stessa posizione. Questa operazione risulta particolarmente utile come fase preliminare alla magnificazione del video, poiché consente di ridurre la presenza di elementi dinamici indesiderati che potrebbero compromettere la qualità del risultato finale.

La magnificazione video è un altro risultato importante delle implementazioni fornite dal progetto. La classe *PhaseMagnificationFrameProcessor* si occupa di applicare un algoritmo di *phase-based video magnification* sviluppato dai ricercatori del MIT (per informazioni più dettagliate si rimanda a [Wad+13]); questa classe si configura come un adapter logico per collegare concretamente il componente di frame processing all'algoritmo di magnificazione salvato nel progetto, in modo da "immergerlo" nella struttura della pipeline. La phase magnification non è altro che una procedura per rendere "visibili" all'occhio umano dei micromovimenti che solo una telecamera può percepire; questo metodo è utile in molti campi, ad esempio nello studio dei micromovimenti strutturali di palazzi ed abitazioni, per stimare la sicurezza e la stabilità della struttura in esame, oppure in medicina, per osservare al meglio sottili movimenti di parti del corpo umano. L'algoritmo comunque prevede, per ottimizzare la precisione del risultato, di caricare in memoria tutto il video (quindi tutti i frame) per poter ottenere un artefatto che tenga conto di tutti i movimenti complessivi nel tempo; per questo motivo la sua efficienza, soprattutto in termini di velocità, diminuisce drasticamente all'aumentare della lunghezza del video usato.

3.2.2 Estrazione, pulizia e analisi dei segnali

Parte forse più importante della catena di elaborazione video è quella di elaborazione dei dati ricavati. Ricordiamo che il framework trasforma i frame video in segnali semantici attraverso una pipeline a tre livelli: Gli *extractor* convertono ogni frame in segnali, i *cleaner* migliorano stabilità e qualità del segnale, mentre gli *analyzer* traducono tali campioni in misure interpretabili dal dominio applicativo.

Nell'implementazione concreta abbiamo dato maggior spazio ai *Signal extractor* basati sul tracking di oggetti. Gli extractor basati su questo contesto sono *OpenCVBufferedSignalExtractor* e *OpenCVStreamSignalExtractor*. Entrambi inizializzano un tracker OpenCV su una ROI (regione di interesse, in genere un rettangolo rappresentato da una bounding box (x, y, w, h)) iniziale, e aggiornano la posizione dell'oggetto nei frame successivi. Sono supportati tracker guidati da algoritmi come *CSRT*, *KCF*, *MIL* e *GOTURN*. Per ogni frame viene prodotto un *BoxSignalSample*, contenente indice del frame, bounding box corrente, centroide e timestamp opzionale. La variante buffered

restituisce un segnale materializzato, mentre la variante stream scrive progressivamente i campioni in uno *SignalBuffer*, risultando adatta a scenari realtime.

Per il tracking multi-oggetto, *OpenCVMultiObjectSignalExtractor* parte da una ROI seme selezionata manualmente. Dal template della ROI iniziale la classe può espandere automaticamente il tracciamento ad altri oggetti simili tramite template matching, fino a un numero massimo configurabile. Ogni oggetto viene seguito da un tracker dedicato e identificato con un *track_id*; per ciascun frame il sistema registra bounding box e centroide in un *MultiObjectSignalSample*. L'extractor emette inoltre eventi *track_created* quando nasce una nuova traccia e *track_lost* quando un tracker non riesce più ad aggiornare l'oggetto.

Il tracking non è chiaramente l'unica possibile fonte di dati. Esistono anche extractor basati su optical flow, che descrivono invece il movimento tra frame consecutivi come un campo vettoriale. *OpenCVSparseOpticalFlowSignalExtractor* usa l'algoritmo di Lucas-Kanade: seleziona punti caratteristici, opzionalmente dentro una ROI, e ne segue le traiettorie nel tempo. Il campione risultante contiene punti, vettori di spostamento per punto, vettore medio globale, magnitudine e angolo. *OpenCVDenseFarnebackSignalExtractor* usa invece l'algoritmo Farneback per calcolare un campo di moto denso, ossia formato da molti punti; il frame viene aggregato in una griglia di celle, ciascuna con un vettore medio, da cui vengono ricavati anche un vettore globale, una magnitudine e un angolo complessivi. Tali segnali sono utili per rilevare un flusso di movimento medio di un certo filmato (come ad esempio un movimento di una piazza di persone).

ArucoMarkerSignalExtractor è un'altra classe di estrazione segnali che rileva marker ArUco; questi marker sono conosciuti ed usati nell'ambito della computer vision per risaltare dei punti particolari in un certo video, usandoli quindi come punti di riferimento. Ogni osservazione include id del marker, corner, centro, stato di detection e quality score basato su area, distanza dal bordo e regolarità della forma. I campioni prodotti sono contenuti dentro a *ArucoMarkerSignalSample*.

Un'altra classe molto interessante è la *YOLOSkeletonCOCOStreamSignalExtractor*, che si occupa di visionare un video, cercare un soggetto umano in movimento e tracciare uno "scheletro" elementare formato da alcuni punti chiave uniti da segmenti, in un formato chiamato *COCO*. Esso usa un modello YOLO pose per estrarre i 17 keypoint dello scheletro COCO; per ogni frame produce coordinate dei keypoint, confidence per punto dell'articolazione e un centroide stabile calcolato a partire dai fianchi. Opzionalmente può includere anche il frame originale nei metadata. L'output è un *COCOSkeletonSignalSample*, pensato per analisi successive su postura e movimento.

La pulizia del segnale è affidata a cleaner specializzati. *MovingAverageCleaner* e *MovingAverageStreamCleaner* smussano i centroidi con una media mobile, dove il secondo è l'evoluzione streaming dell'algoritmo del primo. Un'altra classe è *OutlierRejectionCleaner*, che rimuove, sostituisce o limita valori anomali usando una soglia robusta

basata sul rigetto di valori troppo discordanti dall'insieme dei dati presi (dovuti quindi ad errori occasionali). *SignalWidenerCleaner* amplifica gli spostamenti rispetto alla media del segnale, per garantire una risoluzione migliore grazie all'aumento dell'ampiezza. *OpticalFlowOutlierFilter*, pensato specificatamente per algoritmi di Optical Flow, filtra vettori di flow incoerenti rispetto allo stato precedente. Sono disponibili altri cleaner specifici per determinati signal extractor, come *ArucoTemporalStabilizerCleaner*, che stabilizza nel tempo i marker ArUco con smoothing dipendente dalla qualità e controllo dei salti improvvisi o *COCOSkeletonNormalizationSignalCleaner*, che normalizza gli skeleton COCO centrando il bacino, normalizzando la scala e, opzionalmente, allineando la rotazione.

Gli analyzer per tracking trasformano i segnali in grandezze ricche di significato ed in genere graficabili. Gli analyzer come *HorizontalPositionAnalyzer*, *VerticalVelocityAnalyzer* e simili generano serie temporali delle coordinate posizionali, di velocità o di frequenza, secondo direzioni orizzontali o verticali. Per la velocità si calcolano le derivate rispetto al tempo (o all'indice del frame), mentre per quelli di frequenza si applica una *Trasformata di Fourier Veloce (FFT)* per stimare le componenti oscillanti principali.

Per il tracking multi-oggetto sono disponibili analyzer per la distanza tra coppie di tracce, come *MultipleDistanceAnalyzer*; sono disponibili anche analizzatori che considerano dei particolari segmenti, dette "barriere", osservando quanti oggetti tracciati le attraversano, un'operazione implementata da *MultiObjectBarrierCountingAnalyzer*.

Gli analyzer specializzati operano sui segnali più mirati. *SparseOpticalFlowTrajectoryAnalyzer* ricostruisce traiettorie di punti a partire dal flow sparso, per la ricostruzione del movimento, mentre *DenseOpticalFlowVectorFieldAnalyzer* converte il flow denso in un campo vettoriale visualizzabile. *ArucoMarkerDisplacementAnalyzer* misura lo spostamento assoluto dei marker rispetto alla prima osservazione valida; *ArucoMarkerRelativeMotionAnalyzer* misura invece distanze e variazioni relative tra coppie di marker. Infine, *COCOPoseStreamAnalyzer* analizza gli skeleton COCO normalizzati e li usa per classificare movimenti specifici (per ora basati sui vari movimenti del tennis, come battuta, servizio ecc), preservando al tempo stesso keypoint, confidence e metadata del frame.

3.2.3 Visualizzazione, artifact e riproducibilità

L'output di SEF non è limitato ai risultati numerici prodotti dagli analyzer. Una pipeline può restituire artifact visuali, video annotati, grafici, frame intermedi e metadata di esecuzione, rendendo il risultato più ispezionabile e più facile da riprodurre. Questa scelta separa il dato computato dalla sua rappresentazione: gli analyzer producono oggetti Data, mentre visualizer ed exporter li trasformano in artifact adatti a UI,

debugging o persistenza su file.

Il contratto comune è *VisualArtifact*, un valore immutabile e indipendente dall'interfaccia grafica. Ogni artifact contiene un identificativo stabile, un *kind*, un ruolo semantico, titolo, descrizione e metadata. I sottotipi principali coprono diversi formati, adatti alla rappresentazione di qualsivoglia tipo di dato: *ImageArtifact* rappresenta immagini in memoria, tipicamente in formato PNG; *VideoArtifact* rappresenta video in memoria; *VideoFileArtifact* punta a un video già scritto su disco; *DeferredVideoArtifact* rimanda la generazione del video fino alla materializzazione richiesta dal consumer; *TableArtifact* rappresenta dati tabellari; *JsonArtifact* contiene payload strutturati; *TextArtifact* descrive contenuti testuali, ad esempio con Markdown.

Il ruolo concettuale dell'artifact è espresso da *ArtifactRole*. *final_output* identifica gli output principali destinati all'utente finale; *analysis* rappresenta artifact legati all'interpretazione dei risultati; *debug* raccoglie elementi utili all'ispezione interna della pipeline; *preview* indica output pensati per anteprime o feedback realtime; *diagnostic* segnala artifact prodotti per diagnosi, validazione o troubleshooting. Il ruolo non modifica il contenuto dell'artifact, ma aiuta UI ed exporter a decidere come mostrarlo o archiviarlo.

Durante la generazione degli artifact, i visualizer ricevono un *VisualizationContext*. Questo oggetto trasporta informazioni sul contesto di esecuzione: *pipeline_id*, nome dell'analyzer, nome del visualizer, indice del risultato, metadata della sorgente, metadata di esecuzione e *render.hints*. In questo modo un artifact può essere arricchito con provenienza, parametri e informazioni utili al debugging senza accoppiare il visualizer a una specifica UI.

I visualizer sono divisi in varie categorie, ad esempio molti di loro sono implementati tramite la libreria *Matplotlib*. Essi producono principalmente *ImageArtifact* in formato PNG. *MatplotlibFunctionVisualizer* visualizza funzioni bidimensionali e le grafica, come quelle per posizioni, velocità ecc; *MatplotlibHistogramVisualizer* mostra conteggi categoriali, ad esempio attraversamenti di barriere, tramite un istogramma; visualizer più specifici potrebbero essere *MatplotlibTrajectoryVisualizer*, che disegna traiettorie di punti da optical flow sparso, oppure *MatplotlibVectorFieldVisualizer*, che renderizza campi vettoriali tramite quiver plot, o ancora *MatplotlibHeatmapVisualizer*, che trasforma campi vettoriali in "Heatmap", ossia mappe di magnitudine, componente o angolo. Chiaramente alcuni visualizzatori sono pensati appositamente per determinati analizzatori, ma questa è una necessità che nasce dal fatto che i dati che provengono da certi contesti sono troppo specifici per poter essere trattati in modo astratto: una funzione bidimensionale può essere chiaramente generalizzata, ma i dati vettoriali che provengono, ad esempio, da un optical flow devono essere trattati con metodi più specializzati. Nulla vieta comunque di usare questi visualizzatori per altri dati di formato vettoriale, a patto che rispettino i contratti dichiarati per l'uso di tali componenti.

Per video in realtime, *TrackingVideoVisualizer* produce video annotati con bounding

box, centroidi e label delle tracce, usando artifact file-backed o lazy tramite *DeferredVideoArtifact*. *ArucoAnnotatedVideoVisualizer* estende questa logica al caso ArUco, disegnando corner, centri, id e spostamenti dei marker. *OpenCVCOCOPoseRealtimeVisualizer* mostra skeleton COCO in una finestra OpenCV realtime, mentre *OpenCVCOCOTennisPoseRealtimeVisualizer* aggiunge anche la classe di movimento tennis stimata.

I frame intermedi sono gestiti come artifact diagnostici. *IntermediateFramesVisualizer* produce PNG separati per ogni snapshot intermedio, componendo originale, frame processato, maschere e altre modifiche apportate. *IntermediateFramesGridVisualizer* aggrega più snapshot in una griglia compatta, utile per confrontare rapidamente stadi e frame diversi. In genere questi artifatti servono per vedere visualmente come il processing dei frame modifica effettivamente le immagini, per avere un riscontro effettivo più significativo rispetto a quello dato da semplici dati numerici.

Gli exporter completano la parte di persistenza. *OpenCVFrameBufferVideoExporter* scrive su disco uno stream di frame processati e restituisce un *VideoFileArtifact* finale, mantenendo al tempo stesso un buffer inoltrabile agli stadi successivi. *IntermediateFrameArtifactExporter* esporta invece snapshot intermedi in file PNG, includendo immagine principale, originale, processata e altri dati.

Al termine dell'esecuzione, *PipelineOutputAssembler* costruisce l'oggetto pubblico *PipelineOutputs*. Questo aggrega risultati analitici, artifact finali, artifact di debug, frame intermedi e *PipelineRunMetadata*. Nei metadata vengono inclusi identificativo della pipeline, piano di esecuzione, metriche di latenza e dati di riproducibilità. In questo modo il risultato finale non è solo un insieme di output, ma anche una descrizione ispezionabile di come tali output sono stati prodotti.

La riproducibilità è supportata da un sistema di export dedicato. *PipelineConfigExporter* produce una configurazione contenente la sezione *pipeline* ricostruibile, i descriptor dei componentie i vari metadata. *PipelineExportUtils* fornisce la serializzazione stabile in JSON e YAML, convertendo oggetti come path, date, enum, dataclass e valori NumPy in forme esportabili. Infine, *PipelineCodeExporter* genera codice Python ricostruibile: il codice usa *ConfigPipelineBuilder* e il registry dei plugin per ricreare il *PipelineContext* a partire dalla configurazione esportata. Questo rende possibile ripetere un esperimento, ispezionare una pipeline fallita o condividere una configurazione senza dipendere dallo stato interno dell'esecuzione originale.

3.2.4 SEF Studio come strumento operativo

SEF Studio costituisce il livello di presentazione del framework SEF e ha l'obiettivo di rendere accessibili le funzionalità del sistema anche a utenti che non desiderano costruire una pipeline esclusivamente tramite codice Python o configurazioni dichiarative.

L'interfaccia è stata sviluppata utilizzando Streamlit, un framework Python orientato alla realizzazione di applicazioni web interattive per attività di data science, machine

learning e strumenti tecnici. La scelta di Streamlit è stata motivata dalla possibilità di costruire un'interfaccia completa utilizzando esclusivamente Python, senza introdurre un frontend separato basato su JavaScript o framework esterni.

SEF Studio non implementa un motore di elaborazione indipendente, ma utilizza lo stesso core impiegato dalle API programmatiche e dalle configurazioni dichiarative del framework. Tutte le operazioni di validazione, pianificazione, esecuzione, monitoraggio e gestione dei plugin vengono infatti delegate ai componenti del core, mantenendo una chiara separazione delle responsabilità.

Questo approccio rende l'architettura accessibile a diversi livelli di competenza: un utente inesperto può utilizzare controlli grafici e preset predefiniti, mentre un utente avanzato può intervenire direttamente sulla configurazione JSON o registrare dinamicamente nuovi plugin.

Il flusso concettuale di SEF Studio è illustrato in Figura 3.2.

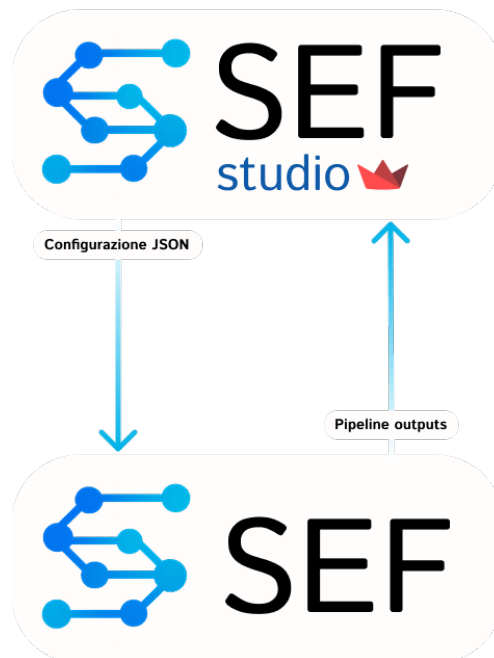


Figura 3.2: Flusso concettuale di SEF Studio.

L'interazione dell'utente viene tradotta in una configurazione dichiarativa della pipeline che viene successivamente validata, trasformata in un *PipelineContext* ed eseguita dal runtime del framework. Il risultato finale viene infine reso disponibile tramite snapshot, eventi e artifact dedicati.

Dal punto di vista architetturale, SEF Studio è organizzato in tre livelli distinti:

- **Presentation layer:** contiene i componenti Streamlit responsabili della costruzione dell'interfaccia grafica.
- **State e view-model:** gestisce la persistenza dello stato della sessione e i modelli tipizzati utilizzati dalla UI.

- **Application services:** traduce le interazioni dell'utente nelle primitive offerte dal core del framework.

Un aspetto fondamentale dell'architettura consiste nel fatto che la dipendenza è unidirezionale: la UI dipende dal core, mentre il core non dipende da Streamlit. Tale separazione permette di mantenere il framework indipendente dalla tecnologia utilizzata per la presentazione.

L'ingresso principale dell'applicazione è rappresentato dal file `ui/app.py`, avviabile tramite il comando:

```
streamlit run ui/app.py
```

L'interfaccia è organizzata in cinque sezioni principali.

Composer Il Composer rappresenta l'ambiente operativo principale e consente di costruire una pipeline tramite strumenti visuali. L'utente può selezionare lo scenario di elaborazione, scegliere i plugin da utilizzare, configurare sorgenti video o webcam, definire regioni di interesse, modificare i parametri dei componenti e generare automaticamente la configurazione JSON corrispondente.

Attualmente sono disponibili due preset completamente operativi:

- *Single Object Tracking*;
- *ArUco Marker Motion*.

Sono inoltre predisposti ulteriori workflow avanzati, attivabili tramite estensione del registry o configurazione manuale:

- Multi-object barrier counting;
- Dense optical flow;
- YOLO pose realtime.

Questa distinzione risulta importante poiché tali workflow dispongono già delle classi applicative e dei controlli grafici necessari, ma non sono ancora registrati come componenti built-in del framework.

Plan & Logs Questa sezione permette di visualizzare il piano di esecuzione senza avviare la pipeline. Vengono mostrati il numero di stage, la distinzione tra elaborazioni streaming e batch, i punti di materializzazione, le impostazioni di buffer e la politica di gestione della latenza.

Una *materialization boundary* rappresenta il punto in cui un flusso progressivo deve essere convertito in una sequenza completa di dati perché uno stage successivo richiede

una modalità batch. Tale operazione può incrementare il consumo di memoria e la latenza iniziale del sistema.

Sono supportate quattro politiche di gestione della latenza:

- *blocking*;
- *drop_newest*;
- *drop_oldest*;
- *adaptive_sampling*.

Run & Monitor Questa sezione è dedicata all'esecuzione e all'osservabilità della pipeline.

Sono disponibili due modalità operative:

- esecuzione sincrona, adatta a elaborazioni offline;
- esecuzione asincrona tramite worker in background.

L'interfaccia mostra lo stato delle pipeline attive, gli snapshot di esecuzione, gli eventi generati dal sistema, i log prodotti dai componenti e gli output persistiti.

I risultati vengono organizzati all'interno della struttura *PipelineOutputs*, che mantiene separati:

- risultati analitici;
- artefatti finali;
- artefatti di debug;
- frame intermedi;
- piano di esecuzione;
- metadati.

Registry La sezione Registry consente di esplorare il catalogo dei componenti disponibili.

Per ogni plugin vengono mostrate informazioni quali categoria, alias, versione, metadati, dipendenze, tag e capacità dichiarate.

È inoltre possibile registrare dinamicamente nuovi componenti specificando il percorso Python di una classe. Tale registrazione è limitata al processo corrente e non viene mantenuta dopo il riavvio dell'applicazione.

Config La sezione Config rappresenta un ambiente indipendente dal Composer e permette di lavorare direttamente sulla configurazione JSON della pipeline.

L'utente può validare la configurazione, eseguire la pipeline in modalità sincrona o asincrona e visualizzare gli output prodotti dal framework.

Un meccanismo di override esplicito evita modifiche accidentali: le modifiche al JSON vengono applicate solo dopo una conferma esplicita dell'utente.

Canvas visuale SEF Studio integra inoltre una rappresentazione grafica della pipeline.

Il canvas visualizza nodi trascinabili che rappresentano i principali gruppi logici della pipeline, tra cui frame extractor, frame processor, signal extractor, signal cleaner, analyzer e visualizer.

Le connessioni tra i nodi sono realizzate tramite porte tipizzate che rappresentano i contratti scambiati tra i componenti.

I tipi supportati sono:

- *frame*;
- *signal*;
- *analysis*;
- *view*;
- *event*.

Durante il trascinamento vengono evidenziate esclusivamente le connessioni compatibili, fornendo una validazione visuale dei contratti.

È importante sottolineare che il canvas non costituisce ancora un editor persistente del DAG della pipeline. Le modifiche effettuate agiscono esclusivamente sullo stato locale dell'interfaccia e non alterano la configurazione JSON del framework.

Editor operativi L'interfaccia integra inoltre strumenti dedicati alla configurazione geometrica delle pipeline.

Tra questi sono presenti:

- selettore dei video;
- ROI selector;
- barrier selector;
- frame overlay editor;
- stage parameter editor;

- editor JSON.

Questi strumenti permettono di configurare pipeline complesse senza intervenire direttamente sul codice.

Supporto realtime SEF Studio implementa inoltre un'infrastruttura browser-compatible per la visualizzazione realtime.

Un servizio dedicato mantiene il frame più recente associato a ciascuna pipeline attiva attraverso strutture thread-safe che adottano una politica *drop-oldest*.

Per la visualizzazione nel browser viene utilizzato uno stream MJPEG erogato tramite un server HTTP locale, evitando la necessità di aprire finestre OpenCV native e consentendo un aggiornamento continuo dell'immagine senza provocare una riesecuzione completa dell'applicazione Streamlit.

Limiti attuali Alcune funzionalità risultano ancora in fase di consolidamento.

In particolare:

- i workflow avanzati non sono ancora registrati come componenti built-in;
- il canvas non costituisce un editor persistente della pipeline;
- il contratto del submit asincrono necessita di un allineamento implementativo;
- SEF Studio non è attualmente distribuito all'interno del package installabile standard del framework.

Nonostante tali limitazioni, SEF Studio rappresenta uno strumento operativo che rende accessibile un'architettura complessa senza sacrificarne l'estensibilità. L'utente può infatti scegliere il livello di astrazione con cui interagire con il framework, passando da preset visuali a configurazioni dichiarative fino all'integrazione diretta di nuovi componenti applicativi.

3.2.5 Modalità d'uso

SEF può essere utilizzato attraverso tre modalità principali, seppur mantenendo la stessa logica sottostante: l'API Python ad alto livello esposta da *import sef*, le configurazioni dichiarative versionate e l'interfaccia grafica *SEF Studio*.

L'API pubblica esposta da *import sef* fornisce una sistema ad alto livello per costruire pipeline in modo fluido: *sef.pipeline(...)* crea una *PipelineFacade* generica, mentre *sef.video(...)* e *sef.webcam(...)* inizializzano direttamente sorgenti video o webcam basate sui plugin built-in. La pipeline viene poi composta tramite metodi dichiarativi: *frames* imposta la sorgente dei frame, *process* aggiunge processori, *signals* definisce l'estrazione del segnale, e via dicendo. La facade espone anche *runtime*, per configurare

buffer e policy di latenza, e *intermediate_frames*, per abilitare la cattura diagnostica degli stadi intermedi. Il metodo *run* costruisce internamente il *PipelineContext* ed esegue la pipeline, mentre *to_config*, *build_context* ed *execution_plan* permettono di ispezionare la configurazione e il piano prima dell'esecuzione.

La stessa facade è progettata per favorire estendibilità e adattabilità. Ogni stadio può essere indicato tramite il nome di un plugin già registrato, una classe, un'istanza già costruita oppure una semplice funzione Python. Quando il riferimento non è una stringa, *StageRegistry* lo registra automaticamente nel registry locale della facade e lo converte in una voce di configurazione compatibile con il core. Le funzioni vengono adattate ai contratti del framework tramite wrapper come *FunctionFrameExtractor*, *FunctionFrameProcessor*, *FunctionSignalExtractor* ecc. Per i casi più semplici, SEF offre anche decorator pubblici: essi registrano funzioni custom come plugin process-local, rendendole immediatamente utilizzabili dalla facade o dalle configurazioni.

Un secondo modo per usare il framework è attraverso *SEF Studio*, un'applicazione Streamlit costruita sopra il runtime del core. Per dettagli più specifici si rimanda alla sezione 3.2.4

Infine, un'ultima modalità d'uso è basata su configurazioni versionate. Una configurazione SEF contiene una sezione *pipeline* con gli stadi principali: *frame_extractor*, *frame_processors*, *signal_extractor*, *signal_cleaners*, *analyzers*, *visualizers*, *runtime* e, opzionalmente, *intermediate_frames*. La funzione *sef.from_config(...)* carica una configurazione esistente e restituisce comunque una *PipelineFacade* eseguibile. A livello core, la configurazione viene validata e normalizzata da *ConfigPipelineBuilder*, che risolve i nomi dei componenti tramite *PluginRegistry* e costruisce un *PipelineContext*. In questo modo API Python, JSON, YAML e configurazioni esportate convergono tutte sullo stesso meccanismo di costruzione.

4. Tecnologie utilizzate

Il framework SEF è stato sviluppato utilizzando un insieme di tecnologie Python e strumenti esterni selezionati in funzione delle diverse responsabilità architetturali del sistema. La scelta delle dipendenze non è stata trattata come un aspetto puramente implementativo, ma come parte della progettazione del framework: il core mantiene infatti un insieme ridotto di dipendenze essenziali, mentre le funzionalità specialistiche, come tracking video, pose estimation, motion magnification e interfaccia grafica, sono collocate in componenti separati o gruppi opzionali.

Questa organizzazione permette di ridurre l'accoppiamento tra il nucleo del framework e le librerie esterne, mantenendo SEF più modulare, estendibile e facilmente adattabile a scenari differenti.

4.1 Gestione modulare delle dipendenze

SEF adotta una gestione modulare delle dipendenze. Il nucleo del framework contiene principalmente le astrazioni di dominio, i contratti dei componenti, il modello dati, il sistema di pipeline, il registro dei plugin e le strutture necessarie all'esecuzione batch e streaming. Le librerie specialistiche non sono quindi distribuite uniformemente all'interno del core, ma sono associate a componenti specifici.

Questa scelta consente di distinguere tra dipendenze essenziali e dipendenze opzionali. Le prime sono necessarie per il funzionamento generale del framework, mentre le seconde vengono utilizzate solo quando si attivano specifiche funzionalità, come l'elaborazione video tramite OpenCV, la stima della posa tramite YOLO, la visualizzazione tramite Matplotlib o l'interfaccia grafica tramite Streamlit.

Un aspetto importante di questa impostazione è il caricamento differito dei componenti. Le tecnologie esterne vengono utilizzate principalmente nei moduli built-in e negli adapter infrastrutturali, mentre il core rimane indipendente dalle implementazioni concrete. In questo modo è possibile sostituire o aggiungere nuovi componenti senza modificare le regole centrali del framework.

4.2 Librerie per l'elaborazione video

4.2.1 OpenCV

OpenCV rappresenta la principale libreria utilizzata da SEF per l'elaborazione video e per molte funzionalità di computer vision. Nel progetto viene utilizzata la distribuzione `opencv-contrib-python`, che include anche moduli aggiuntivi non presenti nella versione base, come alcuni tracker e funzionalità avanzate.

OpenCV viene impiegata per diverse operazioni fondamentali: acquisizione di frame da file video o webcam, scrittura di video in output, lettura di proprietà come frame rate e risoluzione, conversione tra spazi colore, ridimensionamento, rotazione, equalizzazione dell'istogramma, filtraggio, morfologia matematica, background subtraction, optical flow, rilevamento di marker ArUco e inpainting.

Nel framework, OpenCV non è trattata come una dipendenza diretta del core, ma come una tecnologia infrastrutturale utilizzata dai componenti built-in. Ad esempio, gli extractor basati su OpenCV leggono i frame da una sorgente video e li convertono in oggetti `Frame`; gli exporter serializzano un `FrameBuffer` in un file video; i processor applicano trasformazioni o tecniche di pulizia; i signal extractor trasformano l'informazione visiva in campioni di segnale.

Questa separazione permette di mantenere il dominio del framework indipendente dalla specifica libreria utilizzata per la computer vision. OpenCV diventa quindi un adapter operativo, non una regola architetturale del sistema.

4.2.2 NumPy

NumPy costituisce la base numerica del framework. I frame elaborati da SEF sono rappresentati come array multidimensionali, normalmente nella forma:

$$(H, W, C)$$

per immagini a colori, dove H indica l'altezza, W la larghezza e C il numero di canali cromatici. Nel caso di immagini in scala di grigi, la rappresentazione assume invece la forma:

$$(H, W)$$

Questa rappresentazione è particolarmente adatta all'elaborazione video, poiché consente di eseguire operazioni vettorializzate efficienti sui pixel, evitando cicli espliciti Python quando possibile. NumPy viene utilizzata per il calcolo di differenze tra frame, medie, mediane, deviazioni, maschere binarie, coordinate, campi di movimento, skeleton normalizzati e trasformazioni numeriche necessarie agli analyzer.

Un ulteriore vantaggio deriva dal fatto che OpenCV utilizza direttamente array NumPy come rappresentazione delle immagini. Di conseguenza, i frame letti tramite Open-

CV possono essere integrati nel modello dati di SEF senza conversioni strutturali complesse. Questo riduce il costo di adattamento tra libreria esterna e rappresentazione interna del framework.

4.3 Modelli di computer vision

4.3.1 YOLO Pose

Per la stima della posa umana, SEF utilizza modelli YOLO Pose tramite la libreria *ultralytics*. Questa tecnologia permette di rilevare una persona all'interno del frame e di stimare un insieme di keypoint corporei secondo lo schema COCO.

Il modello produce, per ogni soggetto rilevato, le coordinate dei punti articolari e un valore di confidenza associato a ciascun punto. SEF converte questi dati in campioni di segnale, associando a ogni frame informazioni come indice temporale, keypoint, confidence, centroide corporeo e metadata.

Nel caso specifico, il centroide del corpo può essere calcolato come punto medio tra anca sinistra e anca destra:

$$c = \frac{h_{\text{left}} + h_{\text{right}}}{2}$$

dove h_{left} e h_{right} rappresentano rispettivamente le coordinate dell'anca sinistra e dell'anca destra.

Ultralytics utilizza internamente PyTorch per l'inferenza dei modelli, ma SEF non dipende direttamente da PyTorch nel proprio codice. PyTorch può quindi essere considerata una dipendenza transitiva della libreria di pose estimation, non una tecnologia direttamente integrata nel core del framework.

4.3.2 COCO Dataset

Nel contesto del progetto, COCO non viene utilizzato come libreria, ma come formato di riferimento per la rappresentazione dello skeleton umano. In particolare, SEF adotta lo schema COCO a 17 keypoint, che include punti come naso, occhi, orecchie, spalle, gomiti, polsi, anche, ginocchia e caviglie.

Uno skeleton umano può essere rappresentato formalmente come:

$$S \in \mathbb{R}^{17 \times 2}$$

dove ogni riga contiene le coordinate bidimensionali di un keypoint. A questa matrice può essere associato un vettore di confidenza:

$$C \in \mathbb{R}^{17}$$

che indica l'affidabilità della stima per ciascun punto.

Prima dell'analisi, lo skeleton può essere normalizzato per ridurre la dipendenza dalla posizione della persona nel frame, dalla distanza dalla telecamera e dalla dimensione apparente del soggetto. Una possibile normalizzazione consiste nel centrare i punti rispetto al bacino:

$$p'_i = p_i - c_{\text{pelvis}}$$

e successivamente normalizzarli rispetto a una scala corporea, ad esempio la lunghezza del torso:

$$p''_i = \frac{p'_i}{s_{\text{torso}}}$$

Questa trasformazione rende i dati più confrontabili tra frame, soggetti e sequenze video diverse.

4.3.3 ArUco

I marker ArUco vengono utilizzati come riferimenti visivi artificiali, facilmente identificabili all'interno di una scena. Attraverso il modulo `cv2.aruco`, SEF può rilevare i marker presenti nel frame, stimarne gli angoli, calcolarne il centro e produrre un segnale temporale relativo alla loro posizione.

Questa tecnologia è particolarmente utile quando si vogliono misurare spostamenti, vibrazioni o micromovimenti, poiché i marker forniscono punti di riferimento geometricamente stabili. Rispetto al tracking generico di oggetti naturali, l'uso di marker artificiali riduce l'ambiguità visiva e migliora la robustezza del rilevamento.

Nel framework, le informazioni estratte dai marker vengono convertite in campioni di segnale contenenti indice del frame, posizione, identificativo del marker, timestamp e metadata. In questo modo anche il tracking ArUco rientra nello stesso modello generale adottato da SEF per gli altri signal extractor.

4.4 Motion magnification

4.4.1 Eulerian

La Eulerian Video Magnification è una tecnica che consente di amplificare variazioni temporali sottili presenti in una sequenza video. A differenza degli approcci basati sul tracking esplicito di punti o oggetti, questo metodo osserva le variazioni locali dei pixel nel tempo, lavorando quindi da una prospettiva euleriana: l'attenzione è posta sulle posizioni spaziali dell'immagine e sull'evoluzione temporale dei valori osservati in tali posizioni.

Il principio generale consiste nel decomporre il video in componenti spaziali e temporali, isolare una banda di frequenze di interesse e amplificare le variazioni contenute

in tale intervallo. La banda temporale può essere descritta come:

$$f_{\min} < f < f_{\max}$$

dove $f_{\min}$ e $f_{\max}$ indicano rispettivamente la frequenza minima e massima considerate.

Questa tecnica può essere utile per rendere visibili fenomeni difficilmente percepibili a occhio nudo, come piccole vibrazioni, oscillazioni o variazioni periodiche. Nel contesto di SEF, la motion magnification è trattata come un processor specializzato, applicabile a una sequenza di frame prima dell'estrazione o dell'analisi del segnale.

4.4.2 Phase-Based

La Phase-Based Motion Magnification è una variante più avanzata della motion magnification, basata sull'analisi della fase locale del segnale video. Invece di amplificare direttamente le variazioni di intensità, questo approccio sfrutta cambiamenti nella fase delle componenti spaziali per evidenziare piccoli movimenti.

Nel framework SEF, l'algoritmo phase-based non viene reimplementato direttamente nel codice Python. Il componente dedicato agisce come wrapper verso un'implementazione esterna MATLAB o GNU Octave. Il flusso operativo può essere sintetizzato nel seguente modo:

$$\begin{aligned} \text{FrameBuffer} &\rightarrow \text{video temporaneo} \rightarrow \text{script MATLAB/Octave} \\ &\rightarrow \text{video amplificato} \rightarrow \text{nuovo FrameBuffer} \end{aligned}$$

OpenCV viene utilizzata come ponte per serializzare i frame in un video temporaneo, leggere il risultato prodotto dall'implementazione esterna e ricostruire il buffer di frame elaborati. L'esecuzione del runtime MATLAB o Octave avviene tramite processi esterni, ad esempio mediante `subprocess`, e richiede che l'ambiente necessario sia installato separatamente.

I principali parametri configurabili includono fattore di amplificazione, frequenza minima, frequenza massima, frequenza di campionamento, tipo di piramide, scala del video, codec e timeout di esecuzione. Questa scelta consente di integrare una tecnica complessa mantenendo separato il codice del framework dall'implementazione algoritmica esterna.

4.5 Interfaccia grafica

4.5.1 Streamlit

Streamlit viene utilizzato per realizzare SEF Studio, l'interfaccia grafica del framework. La scelta di Streamlit è legata alla possibilità di costruire rapidamente applicazioni web

interattive direttamente in Python, senza dover introdurre un frontend separato basato su tecnologie come React o Vue.

SEF Studio permette di configurare pipeline, caricare video, selezionare componenti, impostare parametri, visualizzare risultati, consultare artefatti intermedi e osservare lo stato dell'elaborazione. Streamlit fornisce widget come slider, checkbox, selectbox, multiselect, form, tabelle, metriche, immagini, video player e pulsanti di download.

Per componenti più interattivi, come la selezione di regioni di interesse o la rappresentazione grafica della pipeline, Streamlit può essere esteso tramite componenti HTML e JavaScript personalizzati. In questo modo la UI mantiene la semplicità dello sviluppo Python, ma può integrare elementi visuali più avanzati quando necessario.

4.5.2 Matplotlib

Matplotlib viene utilizzata per generare visualizzazioni statiche a partire dai dati prodotti dagli analyzer. Nel framework può essere impiegata per rappresentare serie temporali, traiettorie, istogrammi, heatmap, campi vettoriali e grafici relativi agli spostamenti estratti dai video.

Nel contesto di SEF, Matplotlib non è usata come semplice strumento di plotting isolato, ma come parte del sistema di visualizzazione basato su artefatti. I grafici prodotti vengono convertiti in immagini o artefatti visuali, che possono essere poi salvati, esportati o mostrati tramite l'interfaccia grafica.

Questa impostazione mantiene separata la fase di analisi dalla fase di presentazione: gli analyzer producono dati strutturati, mentre i visualizer si occupano della loro rappresentazione grafica. Tale separazione è coerente con l'architettura modulare del framework.

4.6 Strumenti di supporto

4.6.1 Ruff

Ruff è utilizzato come strumento di analisi statica e formattazione del codice. Il suo ruolo è individuare errori comuni, import inutilizzati, problemi di stile e possibili bug prima dell'esecuzione del programma.

Nel progetto, Ruff contribuisce alla qualità del codice perché automatizza controlli che altrimenti sarebbero manuali e soggetti a incoerenze. L'utilizzo di un linter è particolarmente importante in un framework modulare, dove molti componenti devono rispettare interfacce comuni e mantenere uno stile uniforme.

Ruff può inoltre sostituire in parte strumenti separati come flake8, isort e Black, semplificando la configurazione dell'ambiente di sviluppo.

4.6.2 PyTest

Pytest è il principale strumento utilizzato per il testing automatico. I test permettono di verificare il comportamento dei componenti del framework, il corretto funzionamento delle pipeline, la gestione degli errori, il registro dei plugin, l'esecuzione batch e streaming e l'integrazione tra componenti.

L'utilizzo di test automatici è essenziale per un progetto come SEF, perché la modularità introduce molte combinazioni possibili tra extractor, processor, analyzer e visualizer. Attraverso pytest è possibile controllare che l'aggiunta o la modifica di un componente non comprometta il funzionamento delle parti già esistenti.

4.6.3 PyYAML

PyYAML viene utilizzata per il caricamento di configurazioni YAML. Questo permette di descrivere pipeline e parametri in modo dichiarativo, separando la configurazione dal codice Python.

L'uso di file YAML rende il framework più accessibile e riproducibile: una pipeline può essere salvata, modificata, condivisa ed eseguita nuovamente senza riscrivere codice. Questa caratteristica è particolarmente utile negli scenari sperimentali, dove è importante poter documentare con precisione quali parametri sono stati utilizzati in una determinata esecuzione.

4.6.4 Git e GitHub

Git è stato utilizzato come sistema di versionamento del codice, mentre GitHub ha fornito il repository remoto per la collaborazione, la revisione e la distribuzione del progetto.

Il versionamento è fondamentale in un progetto software modulare perché consente di tracciare l'evoluzione dell'architettura, gestire branch di sviluppo, documentare modifiche rilevanti e mantenere una cronologia delle decisioni implementative. GitHub ha inoltre permesso di organizzare il progetto, pubblicare la documentazione, gestire il codice sorgente e predisporre il framework per una distribuzione più ampia.

4.6.5 MkDocs

MkDocs è stato utilizzato per la generazione della documentazione tecnica del framework. A partire da file Markdown, MkDocs consente di produrre un sito statico navigabile, adatto a descrivere installazione, utilizzo, architettura, API e componenti disponibili.

L'integrazione con strumenti come mkdocstrings permette inoltre di generare automaticamente sezioni della documentazione a partire dalle docstring presenti nel co-

dice Python. Questo riduce la distanza tra codice e documentazione, favorendo una maggiore coerenza tra implementazione effettiva e descrizione tecnica del framework.

5. Casi d'uso, valutazione sperimentale

In questo capitolo forniremo una descrizione di come il framework può essere usato, presentando i vari esempi implementati nel progetto al fine di spiegare come le potenzialità di questo framework possano essere sfruttate in casi d'uso reali e concreti.

5.1 Video Tracking

Il tracking rappresenta uno dei casi d'uso più ricchi e naturali per SEF, perché mette insieme molte parti del framework. È stato il primo caso d'uso effettivamente implementato, ed essendo stato presente sin dal prototipo iniziale di SEF, si è arricchito nel tempo di molte caratteristiche interessanti. Nel framework il movimento degli oggetti viene modellato come una sequenza di campioni indicizzati nel tempo: ogni frame produce una bounding box (che non è altro che una tupla a 4 interi *int* che rappresenta un rettangolo), con altri dati rilevanti associati. Questo rende possibile studiare la posizione dell'oggetto, la sua velocità, eventuali frequenze di oscillazione e, nel caso multi-oggetto, relazioni spaziali tra più tracce.

5.1.1 Tracking a singolo oggetto

Il tracciamento single-object è implementato principalmente da *OpenCVBufferedSignalExtractor* e *OpenCVStreamSignalExtractor*. Entrambi partono da una ROI iniziale, cioè una bounding box (x, y, w, h) fornita dall'utente o da un selettore esterno, e inizializzano un tracker OpenCV con algoritmi parametrizzabili a scelta tra *CSRT*, *KCF*, *MIL* o *GOTURN*. Per ogni frame successivo il tracker aggiorna la posizione dell'oggetto e calcola il centroide della bounding box, producendo un *BoxSignalSample*. L'unica differenza risiede nella gestione dell'output: la versione buffered restituisce un segnale completo a fine elaborazione, mentre la versione streaming scrive i campioni in un *SignalBuffer*, rendendola più adatta a pipeline realtime.

Il segnale può essere stabilizzato con cleaner come *MovingAverageCleaner* o *Outlier-RejectionCleaner*, quindi analizzato con gli analyzer di posizione, velocità e frequenza orizzontale o verticale. Se si vuole ispezionare visivamente il risultato, *TrackingPlay-*

backAnalyzer converte i campioni in annotazioni per frame e *TrackingVideoVisualizer* genera un video annotato con bounding box, centroide e identificativo della traccia.

Questo scenario permette di studiare nel dettaglio il movimento di un singolo oggetto: ad esempio la traiettoria di una palla, lo spostamento di un riferimento su una struttura, oppure il movimento di un elemento meccanico. C'è da dire che questo tracking, ad oggi, nella sua implementazione standard nativa, non presenta un singolo caso d'uso specifico, per via del fatto che le sue caratteristiche lo rendono adatto a molte situazioni; l'uso finale va come sempre deciso da chi utilizza SEF.

5.1.2 Tracking multi-oggetto

Il tracking multi-oggetto è gestito dalla classe *OpenCVMultiObjectSignalExtractor*. In questo caso l'utente seleziona una ROI iniziale, che viene usata sia per inizializzare il primo tracker sia come template per cercare altri oggetti simili nel primo frame. Quello che si fa è ricercare istanze simili al primo oggetto, lungo tutta l'immagine. L'extractor può quindi espandere automaticamente le tracce tramite template matching, assegnando a ogni oggetto un *track_id*. Ogni frame produce un *MultiObjectSignalSample*, contenente la lista delle tracce attive con bounding box e centroide. Il componente emette anche eventi *track_created* e *track_lost*, utili per monitorare la nascita o la perdita di oggetti durante l'esecuzione.

Questo caso d'uso è particolarmente interessante perchè permette di osservare non solo singole traiettorie, ma anche relazioni tra oggetti. SEF include analyzer per trasformare queste tracce in informazioni più alte: *MultiObjectBarrierCountingAnalyzer* conta attraversamenti di barriere definite dall'utente, mentre gli analyzer di distanza possono essere usati per misurare la distanza nel tempo tra coppie di oggetti tracciati. Anche i dati multi-oggetto possono essere convertiti in annotazioni playback-ready tramite *TrackingPlaybackAnalyzer*.

Monitoraggio degli attraversamenti stradali

Un esempio applicativo del tracking multi-oggetto è il monitoraggio di attraversamenti stradali. In questo scenario gli oggetti tracciati possono rappresentare automobili, biciclette o altri elementi in movimento. L'utente definisce una o più barriere come segmenti nel piano immagine, ad esempio una linea virtuale all'ingresso di una strada o di una corsia. *MultiObjectBarrierCountingAnalyzer* confronta la posizione precedente e corrente del centroide di ogni *track_id* e verifica se il segmento di movimento interseca una barriera.

Il risultato è un conteggio per categoria: ogni barriera può rappresentare una direzione, una corsia o una zona di interesse. L'analyzer conta gli oggetti unici che attraversano ciascuna barriera almeno una volta, evitando di incrementare ripetutamente il conteggio per lo stesso oggetto sulla stessa linea. In questo modo SEF può derivare

informazioni come numero di passaggi, distribuzione tra direzioni diverse e totale degli attraversamenti osservati nel video.

Questa implementazione può essere usata, per fare degli esempi pratici, ad esempio per studiare come si distribuisce il traffico in un certo orario della giornata in un determinato incrocio stradale, oppure quante persone entrano in una serie di negozi osservati nell'arco di una giornata, e via dicendo.

Analisi sulla stabilità degli edifici

Un altro caso d'uso riguarda l'uso di questa tecnologia multi-tracking per lo studio analitico della stabilità di edifici, muri o altre strutture architettoniche simili. le ROI in questo caso fanno riferimento a punti specifici dell'edificio da analizzare, al fine di osservare come questi punti si muovono nel tempo, come oscillano, quanto varia la loro distanza, e altre informazioni simili.

In SEF questo scenario può essere costruito combinando tracking multi-oggetto, cleaner sui centroidi e analyzer di distanza o frequenza. La distanza tra due punti tracciati permette di osservare deformazioni relative, mentre gli analyzer di posizione e velocità permettono di studiare movimenti lungo gli assi dell'immagine. Il framework non implementa un modello ingegneristico di sicurezza strutturale, ma fornisce gli strumenti per estrarre e visualizzare misure utili a un'analisi successiva. Questo caso diventa ancora più interessante se preceduto da motion magnification, che può rendere visibili micromovimenti difficili da osservare nel video originale. Interessante è anche l'uso con gli analyzer delle frequenze; essi riescono ad ottenere, tramite un algoritmo di *Trasformata di Fourier Veloce (FFT)*, uno spettro di frequenze delle oscillazioni presenti tra i punti di riferimento scelti: tramite queste frequenze possiamo stimare l'intero edificio come un complesso oscillatore armonico i cui parametri fisici (come costante elastiche, rigidità o variabili dinamiche ad esempio) possono essere ricavate dalle informazioni fornite dall'elaborazione del framework. Per un'analisi più approfondita di questo metodo di indagine si rimanda a [SS18].

5.2 Optical flow

L'optical flow è un altro modo di estrarre informazione temporale dai frame proposti. A differenza del tracking, del caso precedente, non richiede una ROI iniziale, perché l'algoritmo si basa sul movimento di punti caratteristici tra frame consecutivi. In generale questi movimenti vengono rappresentati come vettori, e anche su SEF abbiamo deciso di mantenere la stessa convenzione. Tali vettori possono essere analizzati e visualizzati come traiettorie, campi vettoriali o heatmap. La parte che abbiamo implementato nel framework riguarda soprattutto l'estrazione del segnale, la conseguente analisi e la visualizzazione dei risultati ottenuti. Questi non sono casi d'uso veri e propri, ma comunque le applicazioni più avanzate, come stabilizzazione video, compressione file

o orientamento tridimensionale dei robot, possono essere costruite sopra questi dati, implementando un analyzer adeguato all'obiettivo prefissato.

Come già detto in precedenza, in questo ambito gli algoritmi di estrazione dei segnali implementati sono due, *Sparse optical flow* e *Dense optical flow*; algoritmi che si riferiscono alle due macroaree in cui si distingue in generale il campo di ricerca dell'optical flow.

5.2.1 Sparse optical flow

Lo sparse optical flow è implementato dalla classe *OpenCVSparseOpticalFlowSignalExtractor*, che usa l'algoritmo di Lucas-Kanade (fornito dalla libreria *OpenCV*). Nel primo frame proposto vengono selezionati i punti caratteristici con *goodFeaturesToTrack* (eventualmente limitati da una ROI iniziale fornita come parametro per avere un risultato più mirato). Nei frame successivi il componente segue tali punti e produce, per ogni campione, le posizioni correnti, i vettori di spostamento e un vettore medio globale.

Questo approccio è utile quando si vuole seguire un insieme limitato di punti significativi: persone in una piazza, veicoli nel traffico, dettagli locali di un corpo, di una mano o di un volto. SEF include alcuni cleaner mirati, come *OpticalFlowOutlierFilter*, che sostituisce vettori troppo incoerenti rispetto allo stato precedente, e *SparseOpticalFlowTrajectoryAnalyzer*, che ricostruisce traiettorie dei punti nel tempo. Le traiettorie possono poi essere visualizzate con *MatplotlibTrajectoryVisualizer*.

5.2.2 Dense optical flow

Il dense optical flow è implementato da *OpenCVDenseFarnebackSignalExtractor*, basato sull'algoritmo Farneback (sempre tratto da *OpenCV*). In questo caso il movimento viene calcolato in modo denso sull'immagine e poi aggregato in una griglia: ogni cella contiene un vettore medio, e l'intero frame produce anche un vettore globale con magnitudine e angolo. Il campione risultante è un *DenseOpticalFlowSignalSample*.

Questo tipo di segnale descrive il movimento complessivo della scena con molti più punti rispetto allo sparse flow. Nel framework può essere analizzato da *DenseOpticalFlowVectorFieldAnalyzer*, che converte il campo in dati visualizzabili, e rappresentato con *MatplotlibVectorFieldVisualizer* o *MatplotlibHeatmapVisualizer*. I casi d'uso potenziali includono lo studio del moto globale in una folla, la stima di direzioni prevalenti, la visualizzazione di aree instabili o tremolanti e, in prospettiva, applicazioni come stabilizzazione o compressione video, o stima del movimento relativo della camera.

5.3 Motion magnification

La motion magnification è uno dei casi più interessanti per mostrare la natura modulare di SEF. Il framework non reimplementa direttamente l'algoritmo di phase-based motion magnification, ma incapsula l'implementazione esterna MATLAB¹ tramite *PhaseMagnificationFrameProcessor*. Questo dimostra il ruolo di SEF come scheletro architetturale: anche un software esterno può diventare uno stadio della pipeline, con la condizione di mantenere gli stessi contratti di input, output, metadata e artifact intermedi. L'algoritmo è stato preso dal codice fornito dai ricercatori del MIT, per dettagli aggiuntivi si rimanda agli articoli [Wu+12] e [Wad+13].

Il processor consuma l'intero *FrameBuffer* (condizione richiesta dall'algoritmo di magnificazione usato), valida che i frame abbiano forma coerente, ricava o riceve il frame rate, esporta temporaneamente il video, invoca il codice esterno *external/phase_mag* e rilegge il video magnificato come nuovo buffer di frame. I parametri includono *magnification_factor*, banda temporale *low_cutoff_hz/high_cutoff_hz*, *sampling_rate_hz*, e altre specifiche per personalizzare l'esecuzione e il risultato. Ogni frame prodotto contiene metadata *phase_magnification*; se configurato, il processor può anche emettere artifact intermedi diagnostici.

Come caso d'uso, la motion magnification può rendere visibili micromovimenti che poi diventano più facili da analizzare con altri componenti SEF. Per esempio, un video di una struttura può essere prima magnificato e poi dato a un tracker per osservare oscillazioni di punti su un edificio. In ambito biomedico, lo stesso principio può servire a evidenziare piccoli movimenti del corpo o variazioni periodiche difficili da percepire nel video originale. Nel progetto questa tecnica resta un frame processor: ciò significa che può essere combinata liberamente con tracking, export video, frame intermedi e visualizzazioni.

5.4 ArUco per displacement e moto relativo

Il caso d'uso di ArUco prevede una pipeline specializzata costruita sugli stessi mattoni generali del framework. la classe *ArucoMarkerSignalExtractor* rileva marker ArUco frame per frame usando una logica simile a quella per il tracking classico, ma chiaramente che dà risultati migliori essendo l'oggetto da tracciare ben definito da un formato conosciuto e già studiato. A differenza del tracking basato su ROI, qui l'identità dell'oggetto è data direttamente dal marker id: questo riduce l'ambiguità, rende più stabile il riferimento visivo e permette di lavorare con punti fisici preparati appositamente per la misura.

¹Visto l'uso di MATLAB, per testare questo caso d'uso è richiesto un account per poter usare questa piattaforma; all'avvio del programma viene chiamato il software di MATLAB, che procederà all'autenticazione dell'utente; dopo questo passaggio, SEF può continuare il suo lavoro.

Ogni frame produce un *ArucoMarkerSignalSample*, composto da osservazioni per marker con id, spigoli, centro, stato di detection e quality score. Il segnale può essere opzionalmente anche stabilizzato da *ArucoTemporalStabilizerCleaner*, che applica smoothing temporale dipendente dalla qualità della detection e limita salti improvvisi quando l'osservazione è poco affidabile. Tra gli analyzer contiamo *ArucoMarkerDisplacementAnalyzer*, che misura lo spostamento di ogni marker rispetto alla prima posizione valida, e *ArucoMarkerRelativeMotionAnalyzer*, che calcola distanze e variazioni relative tra coppie di marker.

I casi d'uso sono simili al tracking multi-oggetto, ma con riferimenti più controllati. Si possono applicare marker a una struttura, a un oggetto meccanico o a un piano sperimentale e misurare spostamenti assoluti o deformazioni relative. La visualizzazione è supportata da *MatplotlibArucoMotionVisualizer*, per grafici di displacement e moto relativo, e da *ArucoAnnotatedVideoVisualizer*, che genera video annotati con id, corner, centro e valori di spostamento.

5.5 COCO pose realtime e classificazione del gesto

Questo caso ci mostra come il framework possa combinare varie elaborazioni ed analisi per fornire un risultato coerente in real-time. *YOLOSkeletonCOCOStreamSignalExtractor* è una classe che usa un modello *YOLO pose* allenato per l'estrazione di 17 keypoint del corpo umano in formato *COCO*, da ogni frame. Questo formato è uno dei più conosciuti ed usati in quest'ambito applicativo, ed è per questo che è stato scelto. In ogni frame il modello riconosce inizialmente la figura umana principale nel video, dopodiché si appresta a tracciare uno scheletro COCO selezionando i suoi punti di riferimento. Il campione prodotto, *COCOSkeletonSignalSample*, contiene coordinate dei keypoint, confidence per articolazione, timestamp e un centroide calcolato a partire dai fianchi. Opzionalmente, il frame originale può essere conservato nei metadata per visualizzazioni successive.

Il segnale però presenta ancora un problema: non è normalizzato, ossia che i punti considerati non sono uniformati rispetto ad uno standard preciso; gli scheletri possono essere "ruotati" o di dimensioni diverse l'uno rispetto all'altro, e questo inficia non poco nella predizione dei risultati. Tuttavia, SEF prevede un meccanismo di normalizzazione grazie al cleaner *COCOSkeletonNormalizationSignalCleaner*, che centra lo scheletro sul bacino, normalizza la scala e allinea la rotazione.

La classificazione è gestita da *COCOPoseStreamAnalyzer*: prende lo scheletro già normalizzato e lo passa a un modello *joblib* per il riconoscimento di figure umane. Nel caso implementato, le classificazioni previste sono movimenti tipici del tennis come *backhand*, *forehand*, *ready_position* e *serve*. Il modello *joblib* è stato allenato proprio su questi casi grazie al dataset *Tennis Player Actions Dataset* (vedi [Wan+24]). Il risultato è un frame data arricchito con la classe stimata, confidence di predizione quando

disponibile, skeleton, centroidi e metadata.

La visualizzazione realtime è supportata da *OpenCVCOCOPoseRealtimeVisualizer*, che disegna keypoint e connessioni COCO, da *OpenCVCOCOTennisPoseRealtimeVisualizer*, che mostra il movimento tennis riconosciuto, e da *RealtimeCOCOPoseFrameVisualizer*, che pubblica frame renderizzati verso un sink realtime disaccoppiato dalla UI. Questo caso d'uso è efficace per mostrare come SEF possa eseguire una pipeline streaming completa, e chiaramente può essere usata in ambito real-time per analizzare i movimenti in una partita di tennis. Cambiando i componenti usati (mantenendo però l'idea logica sottostante) questo caso d'uso può estendersi anche al riconoscimento di movimenti calcistici, oppure, uscendo dall'ambito sportivo, anche al riconoscimento di movimenti sospetti rilevati da una videocamera pubblica, per garantire più sicurezza.

5.6 Altri scenari applicativi

SEF non vuole essere una "raccolta" di casi d'uso, ma la base da cui essi possono essere costruiti. Ci limitiamo quindi ora a descrivere possibili altri scenari di applicazione di questo progetto, sulla base delle classi già implementate e sulla loro potenzialità futura.

Molti dei frame processor implementati non sono stati spiegati con l'importanza che meritavano: tra questi possiamo citare *ColorStabilizationFrameProcessor*, che trasforma il video entrante riducendo le variazioni di colore ed illuminazione tra i frame (molto utile per scopi come la motion magnification, soprattutto se per video di lunga durata); un altro esempio può essere *DynamicObjectRemovalFrameProcessor*, che usa una stima temporale dello sfondo per rimuovere oggetti transitori o variazioni del colore sporadiche (sempre utile per motion magnification).

Altri frame processor relativi alla *background replacement* e *background subtraction* sono stati implementati per permettere la rimozione sicura di elementi indesiderati nel video; nei casi considerati si sostituisce parte dei frame con un'immagine "ripulita", in modo da ottenere un risultato statico nelle zone considerate, oppure si applica un algoritmo di cancellazione per rimuovere un particolare oggetto sostituendolo con altro, sulla base degli elementi e dello sfondo circostante.

SEF supporta inoltre scenari orientati al debugging e alla riproducibilità. Gli intermediate frame permettono di osservare come cambia il video dopo ogni processor; gli exporter possono salvare video processati o snapshot intermedi, mentre gli artifact visuali rendono la pipeline ispezionabile anche fuori dal codice.

6. Benchmark e valutazione prestazionale

L'obiettivo di questo capitolo è valutare l'impatto delle principali scelte architetturali adottate nello sviluppo di SEF, quantificando il comportamento del framework in differenti scenari di esecuzione.

In particolare, sono stati analizzati tre aspetti fondamentali:

- il comportamento delle pipeline in modalità batch, streaming e ibride;
- l'efficacia delle execution policy nella gestione della latenza;
- il costo computazionale introdotto dall'infrastruttura del framework rispetto a un'implementazione diretta.

Per ciascun benchmark sono state raccolte diverse metriche prestazionali, tra cui:

- tempo totale di esecuzione;
- throughput espresso in frame al secondo (FPS);
- utilizzo massimo della memoria (RSS);
- latenza media di elaborazione;
- percentuale di frame elaborati;
- percentuale di frame scartati;
- staleness dell'ultimo frame elaborato.

I risultati riportati rappresentano il valore mediano ottenuto su più esecuzioni indipendenti, al fine di ridurre l'influenza delle fluttuazioni del sistema operativo e delle attività concorrenti in esecuzione sulla macchina utilizzata.

Tutti i benchmark sono stati eseguiti utilizzando un Apple MacBook Air M2 con 8 GB di memoria unificata e Python 3.11.

6.1 Streaming vs Batch

SEF supporta tre differenti modalità di elaborazione: completamente batch, completamente streaming e ibride. Lo scopo di questo benchmark è valutare l'impatto che la strategia di esecuzione ha sulle prestazioni complessive della pipeline.

I risultati mostrano come una pipeline completamente streaming sia in grado di ottenere il throughput più elevato e la minore latenza di elaborazione. Questo comportamento è dovuto all'assenza di fasi di materializzazione intermedia, che consentono ai dati di attraversare progressivamente tutti gli stage della pipeline.

La configurazione ibrida presenta prestazioni intermedie a causa della presenza di una barriera di materializzazione nello stage `signal_extraction`. Tale barriera interrompe il flusso continuo dei dati e richiede la conversione temporanea delle informazioni in una rappresentazione batch prima di proseguire l'elaborazione.

I risultati evidenziano inoltre una significativa riduzione dell'utilizzo della memoria negli scenari completamente streaming, confermando i vantaggi di questo approccio per applicazioni real-time e per l'elaborazione di video di lunga durata.

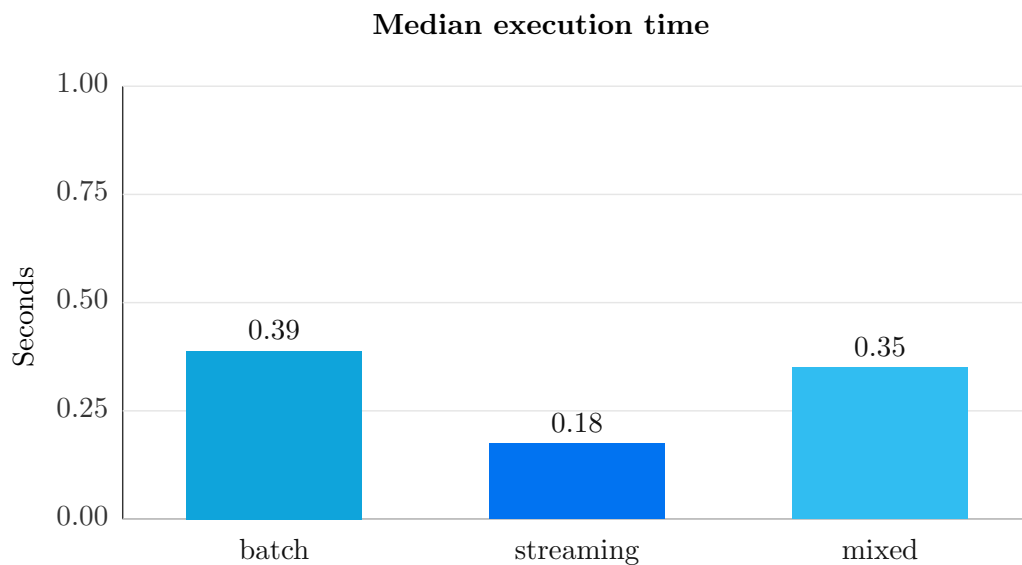


Figura 6.1: Tempo totale di esecuzione nelle diverse configurazioni batch e streaming.

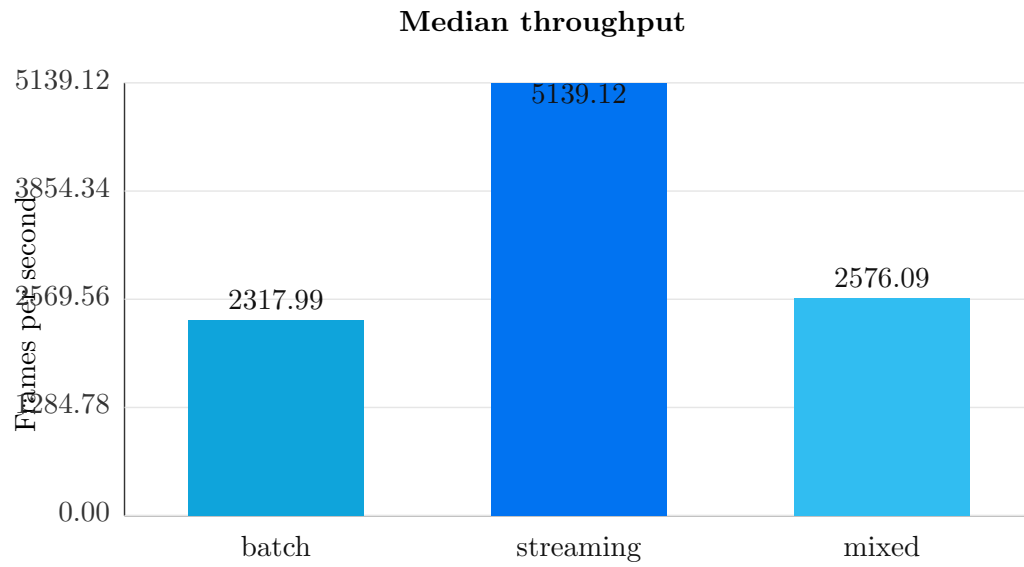


Figura 6.2: Throughput medio espresso in frame al secondo (FPS).

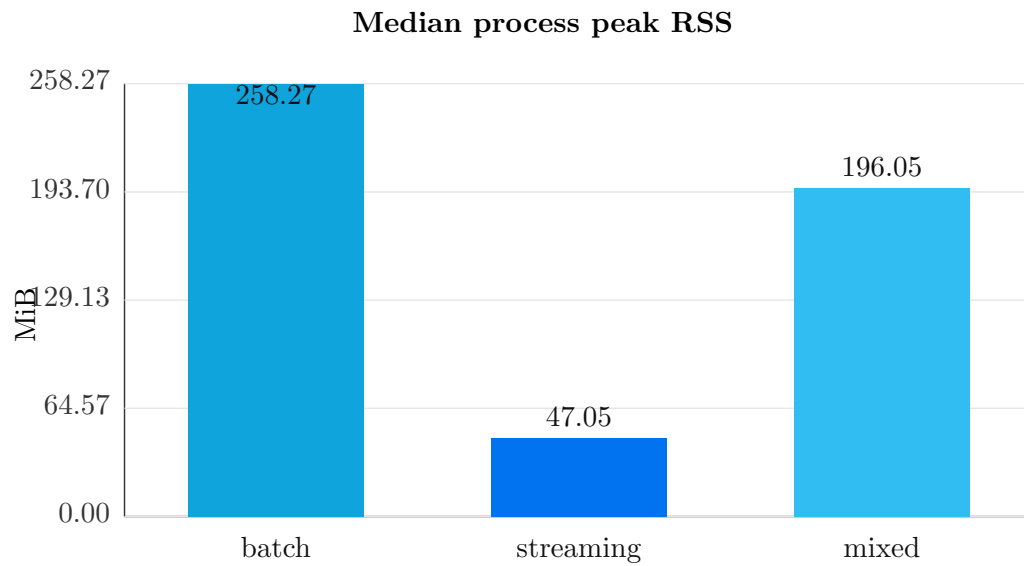


Figura 6.3: Memoria massima utilizzata durante l'esecuzione della pipeline.

6.2 Valutazione delle execution policy

SEF introduce un sistema di execution policy configurabili che consente di controllare il compromesso tra completezza dell'elaborazione e reattività del sistema.

Questa soluzione è resa possibile dall'applicazione del principio di Dependency Inversion. Il runtime non dipende da implementazioni concrete, ma da astrazioni che descrivono il comportamento atteso delle politiche di esecuzione. Di conseguenza, nuove strategie possono essere introdotte senza modificare il motore principale del framework.

La scelta della strategia può essere guidata da differenti esigenze operative, tra cui:

- disponibilità di memoria;
- vincoli di latenza;
- necessità di ottenere risultati progressivi o in tempo reale;
- requisiti di riproducibilità degli esperimenti;
- costo computazionale associato alla materializzazione dei dati.

I risultati mostrano che la policy `blocking` preserva la totalità dei frame prodotti, garantendo la massima riproducibilità dell'elaborazione, ma introduce un tempo complessivo e una latenza media superiori.

Le policy basate sullo scarto o sul campionamento riducono invece drasticamente la latenza e il tempo di elaborazione, accettando una perdita controllata di informazioni.

In particolare, `drop_oldest` ottiene la latenza media più bassa, poiché privilegia i frame più recenti e riduce la staleness dell'informazione visualizzata. La policy `adaptive_sampling` risulta invece la più rapida nel benchmark, introducendo tuttavia una maggiore staleness finale dovuta all'adattamento dinamico dell'intervallo di campionamento.

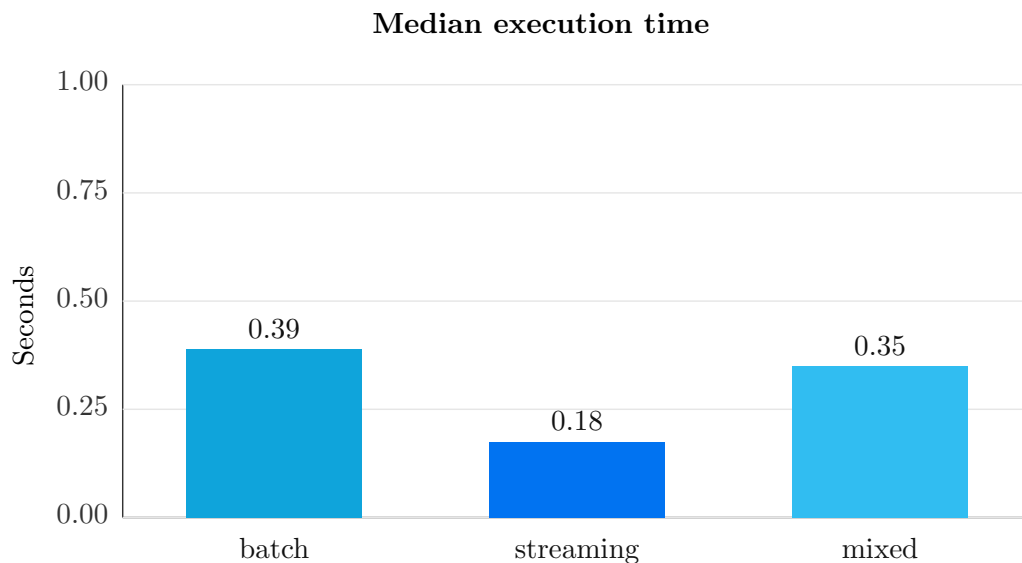


Figura 6.4: Tempo totale di esecuzione delle diverse execution policy.

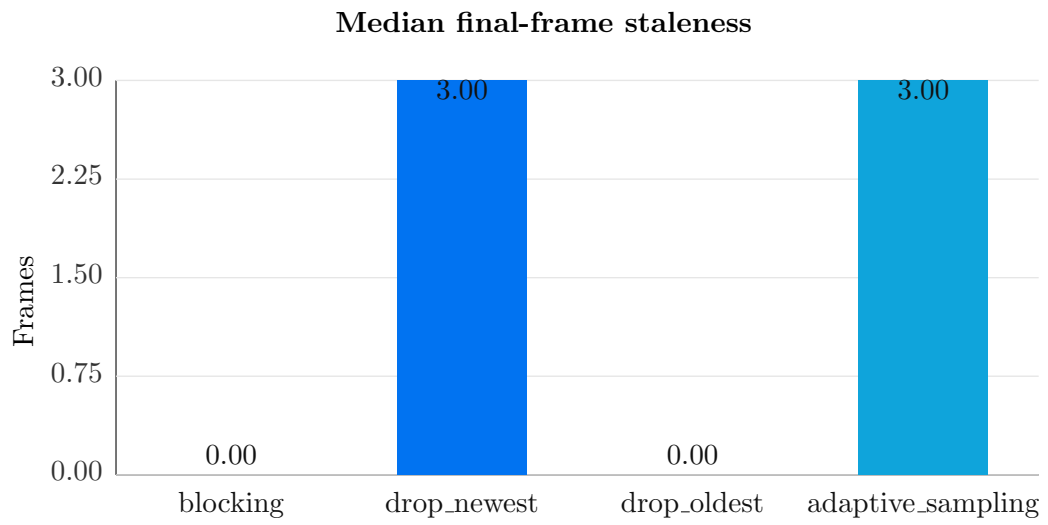


Figura 6.5: Staleness dell'ultimo frame elaborato.

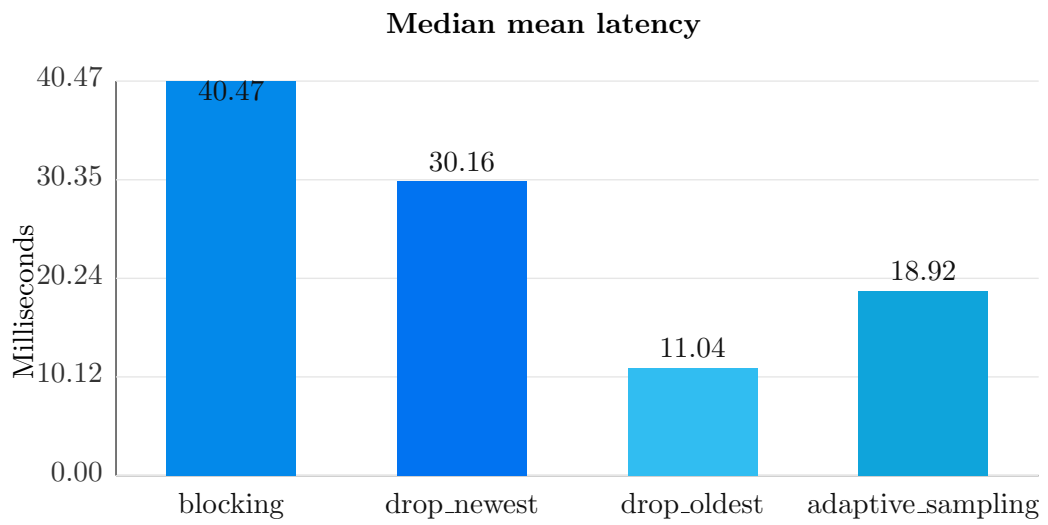


Figura 6.6: Latenza media di elaborazione.

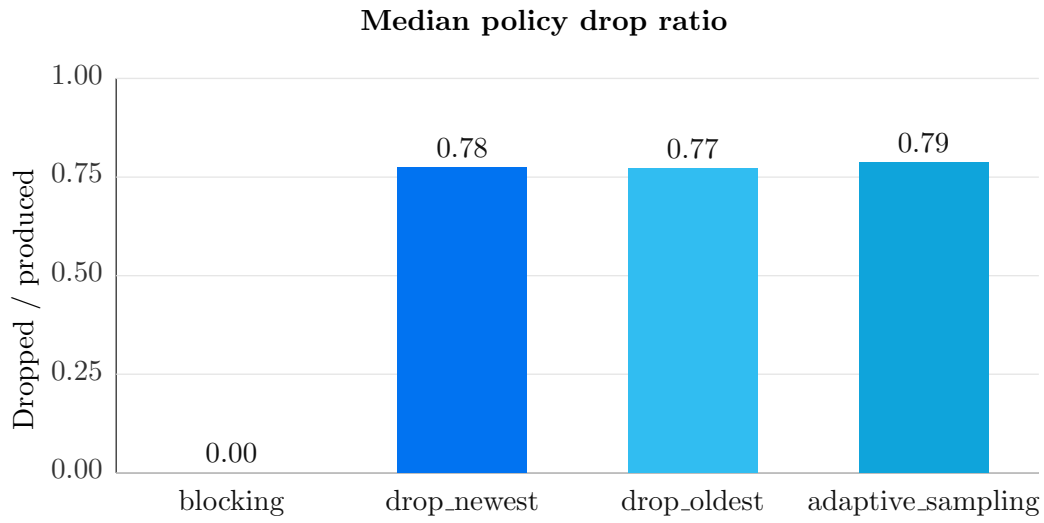


Figura 6.7: Percentuale di frame scartati dalle execution policy.

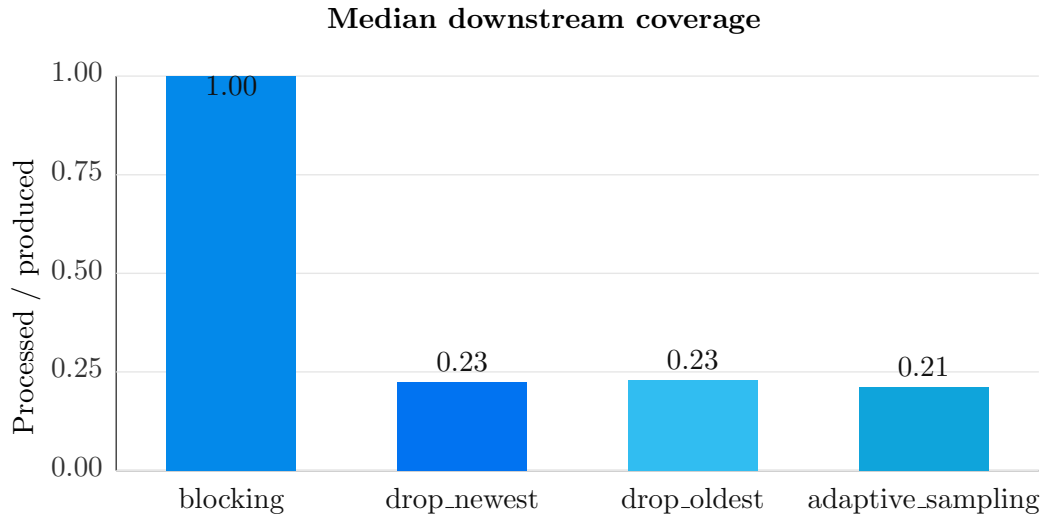


Figura 6.8: Percentuale di frame effettivamente elaborati.

6.3 Overhead architetturale del framework

L'obiettivo di questo benchmark è quantificare il costo computazionale introdotto dall'infrastruttura di SEF rispetto a un'implementazione diretta basata su un semplice ciclo di elaborazione.

I risultati mostrano che il costo introdotto dal framework rimane contenuto. In particolare, SEF introduce un overhead del 10% in modalità batch e del 1% in modalità streaming.

Tale incremento risulta ampiamente compensato dai vantaggi architetturali ottenuti, tra cui:

- modularità;
- osservabilità;
- configurabilità;
- estendibilità;
- riutilizzabilità dei componenti;
- supporto nativo alle pipeline ibride.

Questi risultati dimostrano come le astrazioni introdotte dal framework non rappresentino un ostacolo significativo alle prestazioni complessive del sistema.

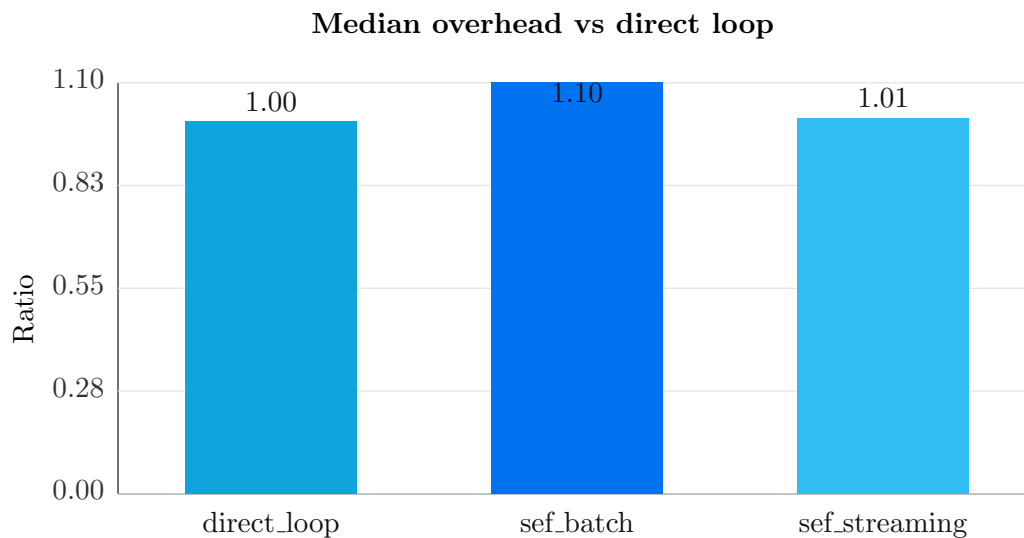


Figura 6.9: Rapporto di overhead rispetto a un'implementazione diretta.

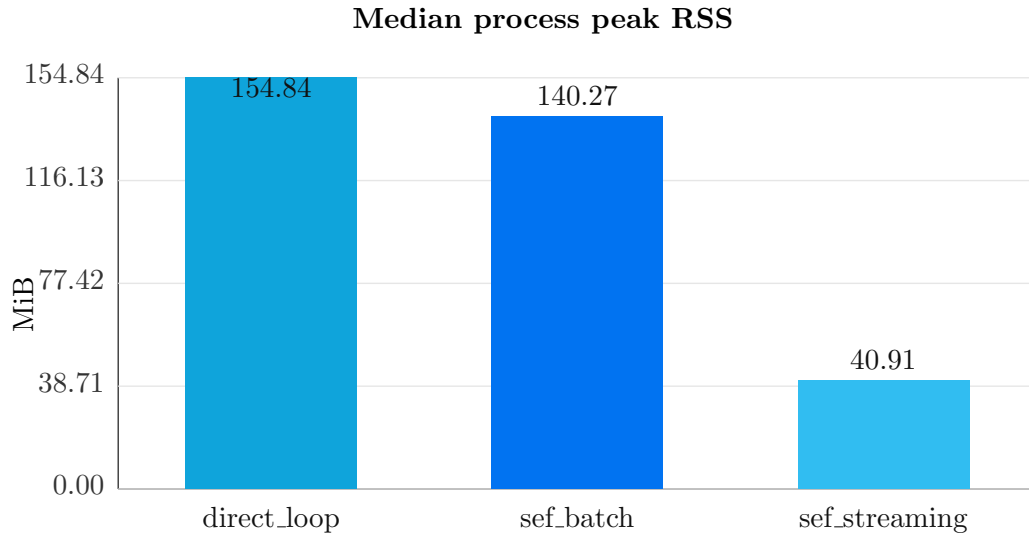


Figura 6.10: Memoria massima utilizzata durante l'esecuzione della pipeline

6.4 Scalabilità in scenari memory-bound

Il benchmark sull'overhead architetturale confronta SEF con un'implementazione diretta costituita da un semplice ciclo di elaborazione. Tale confronto permette di quantificare il costo introdotto dalle astrazioni del framework, ma non descrive completamente il comportamento dei due approcci al crescere della dimensione del problema.

In particolare, uno script sviluppato rapidamente tende frequentemente a materializzare l'intera sequenza video o grandi quantità di dati intermedi in memoria prima di procedere con le fasi successive dell'elaborazione. Questo approccio risulta semplice da implementare e adeguato per video di dimensioni contenute, ma comporta una crescita pressoché lineare dell'utilizzo della memoria all'aumentare della durata del video.

SEF affronta invece questo problema mediante un'architettura progettata fin dall'origine per l'elaborazione streaming. I dati attraversano progressivamente i vari stage della pipeline, vengono mantenuti in memoria solamente per il tempo strettamente necessario e possono essere gestiti tramite buffer a capacità limitata e politiche configurabili di scheduling e materializzazione.

Di conseguenza, all'aumentare della dimensione del dataset, il framework è progettato per mantenere un consumo di memoria sostanzialmente costante, evitando fenomeni di saturazione della memoria principale e il conseguente degrado prestazionale dovuto all'utilizzo della memoria virtuale.

Tale comportamento può portare, in scenari caratterizzati da dataset molto estesi o da limitate risorse hardware, a prestazioni complessive superiori rispetto ad implemen-

tazioni dirette non progettate per gestire tali condizioni, nonostante il lieve overhead architetturale osservato nei benchmark precedenti.

Non viene riportato un confronto con implementazioni manualmente ottimizzate mediante generatori, buffering personalizzato o pipeline concorrenti. La realizzazione di tali ottimizzazioni richiede infatti la progettazione esplicita dell'infrastruttura di elaborazione, rappresentando esattamente la complessità che SEF si propone di incapsulare. L'obiettivo del framework non è sostituire ogni possibile implementazione altamente specializzata, bensì fornire automaticamente un'infrastruttura efficiente e scalabile che permetta allo sviluppatore di concentrarsi esclusivamente sugli algoritmi di computer vision, evitando di dover progettare e mantenere meccanismi di gestione della memoria, orchestrazione della pipeline e controllo del flusso dei dati.

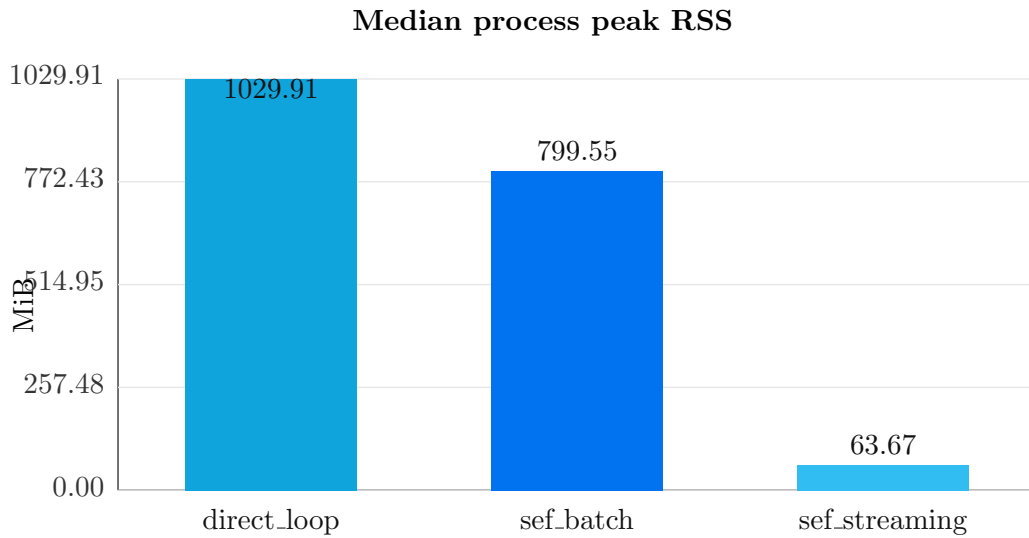


Figura 6.11: Confronto del picco di memoria tra implementazione diretta, SEF batch e SEF streaming.

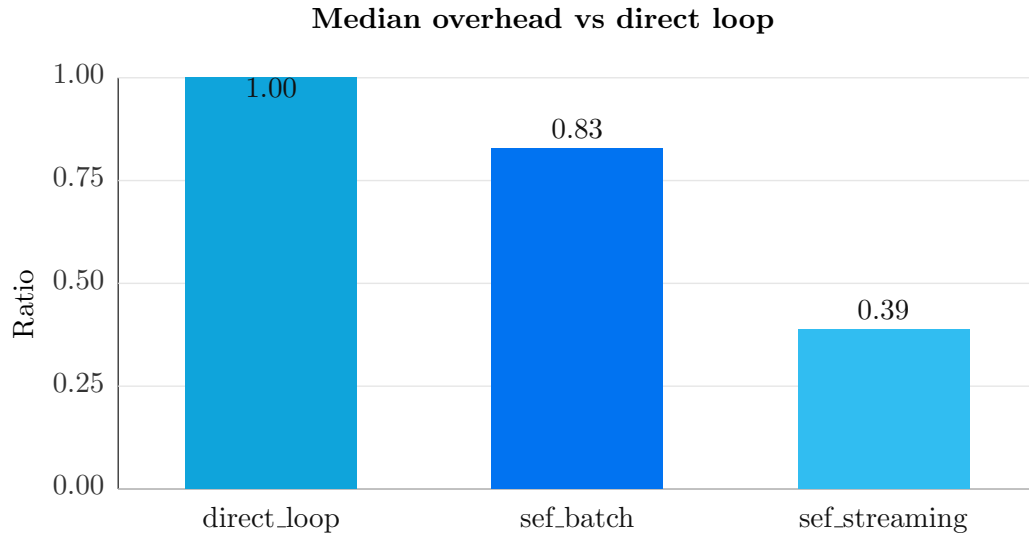


Figura 6.12: Rapporto di overhead rispetto a un'implementazione diretta al crescere della durata del video.

6.5 Discussione dei risultati

I benchmark evidenziano come non esista una singola configurazione ottimale per tutti gli scenari applicativi. Le modalità completamente streaming risultano preferibili quando sono richieste bassa latenza, elaborazione progressiva e ridotto utilizzo della memoria. Le configurazioni batch mantengono invece un ruolo importante nei contesti in cui la riproducibilità dell'esperimento e la disponibilità dell'intero dataset risultano prioritarie.

L'introduzione delle execution policy consente inoltre di adattare dinamicamente il comportamento del framework a differenti vincoli operativi, evitando di imporre una strategia unica di elaborazione.

I risultati ottenuti confermano quindi la validità dell'approccio adottato e dimostrano la capacità di SEF di adattarsi a scenari eterogenei senza richiedere modifiche all'architettura interna.

6.6 Conclusioni

I risultati sperimentali mostrano che SEF soddisfa gli obiettivi progettuali prefissati. Il framework introduce un overhead contenuto, supporta efficacemente modalità batch, streaming e ibride, permette di controllare il compromesso tra completezza e reattività tramite execution policy configurabili e riduce significativamente l'utilizzo della memoria negli scenari streaming.

Tali risultati confermano la validità delle scelte architettoniche adottate e l'adeguatezza del framework per la costruzione di pipeline modulari di analisi video.

7. Discussione finale

7.1 Validità del framework

In questo lavoro è stato proposto SEF, un framework progettato per supportare la costruzione di pipeline modulari dedicate all'analisi di flussi video. La validità della soluzione proposta può essere valutata confrontando gli obiettivi definiti nei capitoli iniziali con i risultati ottenuti durante la progettazione, l'implementazione e la sperimentazione.

I requisiti funzionali e non funzionali introdotti nel Capitolo 2 hanno guidato tutte le principali decisioni architetturali del framework. L'implementazione descritta nel Capitolo 3 mostra come tali requisiti siano stati tradotti in componenti software concreti, mentre i casi d'uso e i benchmark del Capitolo 5 consentono di verificarne l'effettiva applicabilità.

L'architettura proposta ha dimostrato di poter supportare casi d'uso differenti, tra cui *tracking di oggetti*, *optical flow*, *motion magnification*, analisi tramite *marker ArUco* e *pose estimation*, mantenendo invariata la struttura generale della pipeline. Questo risultato suggerisce che la separazione tra core, contratti e implementazioni concrete consenta effettivamente di riutilizzare la stessa infrastruttura indipendentemente dagli algoritmi utilizzati.

Dal punto di vista prestazionale, i benchmark hanno evidenziato come il runtime streaming permetta di ridurre significativamente i tempi di elaborazione e l'utilizzo della memoria rispetto all'esecuzione batch. Inoltre, la valutazione dell'overhead architetturale mostra che il livello di astrazione introdotto da SEF comporta un costo computazionale contenuto, rendendo il framework utilizzabile anche in scenari nei quali le prestazioni rappresentano un requisito importante.

Un ulteriore elemento di validazione deriva dalla capacità del framework di supportare modalità di utilizzo differenti attraverso la stessa architettura. API Python, configurazioni dichiarative, CLI e interfaccia grafica condividono infatti il medesimo modello di pipeline, evitando duplicazioni e garantendo un comportamento coerente indipendentemente dal punto di accesso scelto dall'utente.

Nel complesso, i risultati ottenuti confermano che gli obiettivi definiti all'inizio del lavoro sono stati raggiunti. SEF non si limita a fornire un insieme di algoritmi di computer vision, ma propone un'infrastruttura software modulare, estendibile e riuti-

lizzabile, progettata per separare la logica di composizione delle pipeline dalle tecniche di elaborazione impiegate. Questa scelta architetturale rappresenta il principale contributo del lavoro e costituisce la base per le future evoluzioni del framework. La validità del framework non è quindi dimostrata dal successo di un singolo algoritmo di computer vision, bensì dalla capacità della stessa architettura di supportare algoritmi differenti senza richiedere modifiche al core del sistema. Questo rappresenta uno degli obiettivi progettuali principali definiti all'inizio del lavoro.

7.2 Distribuzione e maturità del progetto

La pubblicazione di SEF su *PyPI* e *GitHub* rappresenta un importante indicatore del livello di maturità raggiunto dal progetto. Diversamente da un prototipo sviluppato esclusivamente a scopo sperimentale, un framework distribuito pubblicamente deve garantire un'API stabile, una gestione coerente delle dipendenze, un sistema di versionamento, una documentazione completa e procedure di installazione riproducibili. La distribuzione del framework costituisce quindi non soltanto una modalità di rilascio del software, ma anche una dimostrazione della volontà di renderlo realmente utilizzabile, estendibile e mantenibile nel tempo.

Per supportare questo obiettivo, SEF è stato progettato come un progetto software completo e non come un semplice insieme di esempi di elaborazione video. Il framework è installabile direttamente tramite il gestore di pacchetti Python utilizzando il comando:

```
pip install sef
```

Questa modalità di distribuzione ne rende immediato l'utilizzo all'interno di nuovi progetti e ne favorisce l'integrazione nei comuni workflow di sviluppo.

Parallelamente alla distribuzione del package è stata realizzata una documentazione tecnica mediante MkDocs e pubblicata tramite GitHub Pages. La documentazione comprende la descrizione dell'architettura, delle API pubbliche, dei principali componenti disponibili e numerosi esempi di utilizzo, con l'obiettivo di semplificare l'apprendimento del framework e favorire lo sviluppo di nuove estensioni e plugin.

Particolare attenzione è stata inoltre dedicata alla progettazione dell'API pubblica. L'interfaccia è stata sviluppata privilegiando semplicità, leggibilità e coerenza, permettendo di costruire pipeline in maniera dichiarativa senza rinunciare, quando necessario, all'accesso ai livelli più avanzati dell'architettura. La presenza di configurazioni versionate in formato JSON e YAML contribuisce inoltre a migliorare la riproducibilità degli esperimenti, la condivisione delle pipeline e la possibilità di automatizzarne l'esecuzione.

SEF mette inoltre a disposizione una *Command Line Interface* (CLI) che consente di validare configurazioni, generare template, eseguire pipeline e ispezionare i componenti disponibili direttamente da terminale. Questa modalità di utilizzo facilita l'integrazione

del framework in workflow automatizzati, sistemi di *Continuous Integration* e contesti nei quali non è disponibile un'interfaccia grafica.

A completare l'ecosistema è presente SEF Studio, un'interfaccia grafica sviluppata mediante Streamlit che permette di configurare, eseguire e osservare le pipeline attraverso strumenti visuali. Sebbene l'implementazione attuale rappresenti ancora un prototipo, essa dimostra come sia possibile costruire strumenti di alto livello sfruttando lo stesso core architetturale utilizzato dall'API Python e dalla CLI. Tra gli sviluppi futuri è prevista la realizzazione di una nuova interfaccia grafica più completa, in grado di offrire funzionalità avanzate per la progettazione, il monitoraggio e il debugging delle pipeline.

Nel complesso, la disponibilità di un package distribuito tramite PyPI, di una documentazione tecnica dedicata, di un'API pubblica stabile, di strumenti da linea di comando e di un'interfaccia grafica evidenzia come SEF sia stato progettato con una prospettiva orientata alla distribuzione e al riutilizzo del software.

7.3 Limiti attuali

Nonostante i risultati ottenuti dimostrino la validità dell'architettura proposta, SEF presenta ancora alcuni limiti che rappresentano interessanti opportunità di evoluzione. Tali limitazioni non compromettono il funzionamento del framework, ma individuano gli aspetti sui quali è possibile estenderne ulteriormente le capacità.

Limiti architetturali

L'attuale architettura del framework supporta esclusivamente l'esecuzione locale delle pipeline. Sebbene il modello adottato separi chiaramente la descrizione della pipeline dalla sua esecuzione, non è ancora presente un'infrastruttura dedicata all'orchestrazione distribuita su nodi remoti, cluster o servizi cloud.

La separazione tra configurazione dichiarativa, orchestratore e meccanismi di esecuzione rende tuttavia possibile introdurre nuove strategie di esecuzione senza modificare il core del framework. Un'evoluzione naturale consiste nell'implementazione di nuovi *PipelineRunner* dedicati all'esecuzione distribuita, capaci di delegare l'esecuzione delle pipeline a differenti nodi di calcolo mantenendo invariato il contratto pubblico del framework.

Uno sviluppo ancora più avanzato potrebbe prevedere la distribuzione dell'esecuzione a livello dei singoli stage della pipeline. In tale scenario ciascuna fase potrebbe essere assegnata dinamicamente al nodo hardware più adatto in funzione delle risorse disponibili e delle caratteristiche computazionali richieste dagli algoritmi impiegati. Ad esempio, componenti che richiedono accelerazione GPU potrebbero essere eseguiti su

nodi dotati di hardware dedicato, mentre operazioni meno onerose potrebbero essere affidate a macchine differenti.

Un'infrastruttura di questo tipo introdurrebbe tuttavia problematiche aggiuntive, quali sincronizzazione tra nodi, tolleranza ai guasti, gestione della latenza di rete, trasferimento dei dati e pianificazione distribuita dell'esecuzione. Per tali motivi questa funzionalità non è stata sviluppata nell'ambito del presente lavoro, privilegiando la definizione di un'architettura modulare sulla quale tali estensioni possano essere costruite in futuro.

Limiti implementativi

SEF Studio rappresenta attualmente un prototipo funzionale che consente di configurare, eseguire e osservare le pipeline attraverso un'interfaccia grafica. Tuttavia il canvas disponibile svolge principalmente una funzione illustrativa e non costituisce ancora un vero editor visuale delle pipeline.

Un'evoluzione significativa consiste nello sviluppo di un editor completo basato su operazioni di *drag-and-drop*, capace di consentire la costruzione, modifica e validazione delle pipeline direttamente dall'interfaccia grafica. L'implementazione attuale è inoltre limitata dalle caratteristiche della libreria Streamlit, progettata principalmente per la realizzazione di dashboard e applicazioni interattive relativamente semplici. Lo sviluppo di un'interfaccia maggiormente evoluta potrebbe richiedere l'adozione di tecnologie dedicate alla realizzazione di applicazioni desktop o web più complesse.

Ulteriori aspetti implementativi riguardano il consolidamento di alcune funzionalità ancora sperimentali, tra cui workflow avanzati, gestione asincrona completa delle esecuzioni e strumenti più evoluti per il monitoraggio e il debugging delle pipeline.

Limiti dell'ecosistema

Attualmente l'ecosistema di plugin risulta ancora limitato, essendo stato sviluppato principalmente con l'obiettivo di validare l'architettura del framework e coprire i principali casi d'uso sperimentali affrontati durante il lavoro.

L'efficacia di un framework modulare dipende tuttavia anche dalla disponibilità di componenti riutilizzabili. Per questo motivo uno degli obiettivi futuri consiste nell'ampliamento del catalogo di plugin ufficiali, nella semplificazione delle procedure di sviluppo di componenti personalizzati e, in prospettiva, nella realizzazione di un marketplace dedicato alla distribuzione e condivisione dei plugin sviluppati dalla comunità.

Limiti sperimentali

La valutazione sperimentale presentata in questa tesi è stata condotta su una singola piattaforma hardware e su un numero limitato di dataset e casi di studio. Sebbene tali

risultati siano sufficienti per verificare la correttezza dell'architettura proposta e valutare il comportamento prestazionale, una validazione più ampia richiederebbe benchmark eseguiti su configurazioni hardware differenti, dataset maggiormente eterogenei e scenari applicativi di complessità superiore.

7.4 Sviluppi futuri e conclusioni

Le limitazioni discusse nella sezione precedente individuano anche le principali direzioni di sviluppo del framework.

Dal punto di vista architetturale, uno degli sviluppi più significativi consiste nell'introduzione di meccanismi di orchestrazione distribuita, capaci di eseguire pipeline su cluster di calcolo, infrastrutture cloud o nodi remoti mantenendo invariata la configurazione dichiarativa utilizzata dall'utente. L'evoluzione dell'attuale modello di orchestrazione potrebbe inoltre consentire l'introduzione di strategie di scheduling dinamico e meccanismi di bilanciamento del carico.

Per quanto riguarda il runtime, risultano particolarmente interessanti l'integrazione di backend dedicati all'accelerazione hardware, il supporto a differenti tecnologie di esecuzione GPU e l'introduzione di politiche di scheduling più evolute, capaci di adattare automaticamente il comportamento della pipeline alle caratteristiche del sistema sul quale viene eseguita.

Un ulteriore ambito di crescita riguarda l'espansione dell'ecosistema del framework. L'ampliamento del numero di plugin disponibili, l'integrazione di nuovi algoritmi di computer vision, l'introduzione di ulteriori analyzer e visualizer e la costruzione di una comunità di sviluppatori rappresentano elementi fondamentali per aumentare il valore pratico del framework e favorirne l'adozione.

Infine, l'integrazione di modelli di intelligenza artificiale costituisce un ulteriore sviluppo di particolare interesse. La crescente diffusione di modelli multimodali e di tecniche di analisi automatica dei contenuti video apre infatti nuove possibilità per la costruzione di pipeline capaci di combinare algoritmi tradizionali di computer vision con modelli di apprendimento automatico mantenendo invariata l'architettura generale del framework.

In conclusione, questa tesi ha portato alla progettazione e allo sviluppo di SEF, un framework modulare per l'analisi e l'elaborazione di flussi video progettato con particolare attenzione alla separazione delle responsabilità, all'estendibilità e alla riusabilità del software. Il contributo principale del lavoro non consiste nella realizzazione di un nuovo algoritmo di computer vision, bensì nella definizione di un'infrastruttura software capace di integrare algoritmi differenti mantenendo separata la logica di composizione delle pipeline dalle tecniche di elaborazione impiegate.

I risultati ottenuti dimostrano come un'architettura basata su contratti, configurazioni dichiarative e plugin consenta di costruire un sistema flessibile, estendibile e riu-

tilizzabile senza introdurre un overhead prestazionale significativo. La pubblicazione del framework su PyPI e GitHub, unitamente alla documentazione tecnica sviluppata durante il progetto, testimonia inoltre la volontà di trasformare SEF da progetto sperimentale a framework realmente utilizzabile dalla comunità.

Pur presentando alcuni limiti, l'architettura definita costituisce una base solida sulla quale costruire future evoluzioni sia in ambito applicativo sia nel contesto della ricerca. In questa prospettiva, SEF rappresenta non un punto di arrivo, ma una piattaforma aperta destinata ad evolvere con l'integrazione di nuovi algoritmi, nuove tecnologie e nuovi scenari applicativi.

Bibliografia

- [Mar17] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. ISBN: 9780134494272.
- [SS18] Zhexiong Shang e Zhigang Shen. «Multi-point vibration measurement and mode magnification of civil structures using video-based motion processing». In: *Automation in Construction* 93 (2018), pp. 231–240. ISSN: 0926-5805. DOI: [10.1016/j.autcon.2018.05.025](https://doi.org/10.1016/j.autcon.2018.05.025).
- [Wad+13] Neal Wadhwa et al. «Phase-Based Video Motion Processing». In: *ACM Transactions on Graphics* 32.4 (2013), 80:1–80:10. DOI: [10.1145/2461912.2461966](https://doi.org/10.1145/2461912.2461966).
- [Wan+24] C.-Y. Wang et al. *Tennis Player Actions Dataset for Human Pose Estimation*. Ver. 1. Dataset disponibile anche tramite Kaggle. 30 Apr. 2024. DOI: [10.17632/nv3rpsxhhk.1](https://doi.org/10.17632/nv3rpsxhhk.1). URL: <https://www.kaggle.com/datasets/orvile/tennis-player-actions-dataset> (visitato il giorno 26/06/2026).
- [Wu+12] Hao-Yu Wu et al. «Eulerian Video Magnification for Revealing Subtle Changes in the World». In: *ACM SIGGRAPH 2012*. 2012.

Ringraziamenti

Desideriamo innanzitutto ringraziare il professor Michele Loreti, nostro relatore, che ci ha affiancato durante tutto il percorso di progettazione e sviluppo del framework e nella stesura di questa tesi. La sua disponibilità, i suoi consigli e il continuo confronto ci hanno permesso non solo di realizzare SEF, ma soprattutto di crescere dal punto di vista tecnico e professionale, insegnandoci ad affrontare un progetto di ricerca e sviluppo con metodo, rigore e attenzione alla qualità.

Un ringraziamento particolare va poi a Vincenzo Fioriti, nostro correlatore, che ci ha guidato nello sviluppo di un caso d'uso reale di interesse nazionale, offrendoci preziosi spunti di riflessione e dimostrando costante interesse verso il lavoro svolto.

Ringraziamo poi l'Università di Camerino per le opportunità, i servizi e il percorso formativo offerto, durante questi tre anni di studio. L'esperienza universitaria ci ha permesso di acquisire solide competenze nell'ambito dell'informatica, ampliare il nostro bagaglio culturale e costruire le basi necessarie per affrontare con consapevolezza il nostro futuro professionale.

Questo lavoro non rappresenta per noi un punto di arrivo, ma piuttosto l'inizio di un nuovo percorso. Le esperienze vissute, le sfide affrontate e le conoscenze acquisite costituiscono il punto di partenza verso nuovi obiettivi, nuove opportunità e una continua crescita personale e professionale.